

2025 – 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY: INFORMATION TECHNOLOGY

**Design of a Multi-Agent AI Pipeline
for Evidence Collection from
Illegal Streaming Sites**

By: Ahmed ARFAOUI

Academic supervisor: Naadine Maazoun

Corporate Internship Supervisor: Wael Hkiri

Soft Stars

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report:

- **Nom & Prénom / Name & Surname** : Ahmed ARFAOUI

Encadrant Entreprise / Business site Supervisor

- **Nom & Prénom / Name & Surname** : Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique / Academic Supervisor

- **Nom & Prénom / Name & Surname** : Naadine Maazoun

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné** / This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde / This form must be inserted after the cover page.

Abstract

This report presents the design, development, and evaluation of an agentic evidence-collection system for illegal streaming websites. The work was carried out during an internship at Soft Stars in the context of sports media protection for beIN SPORTS. The mission was to support rights-enforcement review by visiting target websites, classifying their pages, extracting hosting and stream evidence, and preserving auditable outputs such as screenshots, traces, and structured results.

The internship produced two related artifacts: an exploratory workflow used to validate browser-agent behavior on real targets, and **Open Web Catcher (OWC)**, the complete platform designed to improve maintainability, observability, memory, evaluation, and scalability. OWC was the internal name because the system catches open-web evidence through browser-visible investigation rather than treating the work as a simple scraper. The work did not aim to replace every operational process in production. It focused on proving and structuring a reviewable engineering approach for dynamic streaming websites.

The final architecture organizes evidence collection around specialized reasoning agents, browser automation tools, structured outputs, trace storage, screenshot capture, and evaluation metrics. The database described in this report is limited to observability, evaluation, traces, metrics, screenshots, errors, tool calls, and structured outputs; it is not the enterprise production database of the client organization.

Keywords:. multi-agent system; browser automation; illegal streaming; evidence collection; stream extraction; observability; provider analysis.

Résumé

Ce rapport présente la conception, la réalisation et l'évaluation d'un système agentique de collecte de preuves sur des sites de streaming illégal. Le travail a été effectué durant un stage chez Soft Stars, dans un contexte de protection des droits sportifs pour beIN SPORTS. L'objectif était de rendre l'observation des sites plus traçable: classification des pages, extraction des pages d'hébergement et des flux, capture de captures d'écran, conservation des traces et préparation d'éléments exploitables pour une revue humaine.

Le projet a d'abord utilisé un prototype n8n afin de valider le comportement des agents navigateur sur des sites réels. Les limites observées pendant cette phase ont ensuite guidé Open Web Catcher (OWC), la plateforme complète organisée autour d'agents spécialisés, d'outils navigateur, d'une persistance des traces et d'une évaluation par régression. Ce nom interne reflète l'objectif de capturer des preuves visibles sur le Web ouvert. Le résultat constitue une base technique pour poursuivre l'industrialisation de la collecte de preuves.

Mots clés: système multi-agent; automatisation navigateur; streaming illégal; collecte de preuves; extraction de flux; observabilité; analyse fournisseur.

Acknowledgements

I would like to thank my academic supervisor, Naadine Maazoun, and my professional supervisor, Wael Hkiri, for their guidance throughout this internship. I also thank the Soft Stars team for the technical discussions, feedback, and trust that made this work possible.

Contents

List of Acronyms	xiii
Terminology	xv
1 Introduction	1
1.1 Host Organization and Internship Context	1
1.1.1 Host Organization	1
1.1.2 Operational Context	1
1.2 Problem Statement	1
1.3 Internship Mission and Scope	2
1.3.1 Assigned Mission	2
1.3.2 Mission Deliverables	2
1.3.3 Scope Boundaries	2
1.4 Objectives and Main Contributions	3
1.4.1 Objectives	3
1.4.2 Main Contributions	3
1.5 Report Organization	3
2 Methodology, Requirements, and Design Constraints	5
2.1 Methodological Choices	5
2.1.1 Selection Logic	5
2.1.1.1 Artifact Fit	5
2.1.1.2 Iteration Fit	5
2.1.1.3 Evaluation Fit	5
2.2 Adopted Methodology	6
2.2.1 Backbone and Supporting Lenses	6
2.2.1.1 DSRM Backbone	6
2.2.1.2 Agile Iterations	6
2.2.1.3 CRISP-DM Evaluation Lens	7
2.3 Project Evolution	8
2.3.1 Main Phases	8
2.3.1.1 Exploration and Prototype	8
2.3.1.2 OWC Architecture and Evaluation	8
2.4 Functional Requirements	8
2.4.1 Requirement Groups	8

2.4.1.1	Ingestion and Routing	8
2.4.1.2	Evidence Collection	9
2.5	Non-Functional Requirements	9
2.5.1	Quality Attributes	9
2.5.1.1	Traceability and Observability	9
2.5.1.2	Maintainability and Cost Control	9
2.6	Chapter Summary	9
3	Technical Background	11
3.1	Streaming Website Structure	11
3.2	Browser-Rendered Pages and the DOM	11
3.3	Media Streaming and Manifests	12
3.4	Runtime Obstacles	13
3.5	Agentic Workflows and Tool Use	13
3.6	Evidence Traceability	14
4	System Architecture	15
4.1	Use-Case View	15
4.2	Component Architecture	15
4.3	Agent Pipeline	16
4.4	Routing and Handoff Rules	17
4.5	Agent Runtime Class View	17
4.6	Data and Evidence Persistence	17
4.7	Chapter Summary	17
5	Implementation	21
5.1	Prototype Phase with n8n	21
5.1.1	Prototype Goal	21
5.1.2	Prototype Findings	21
5.1.2.1	What Worked	21
5.1.2.2	What Failed	21
5.2	Open Web Catcher Platform	23
5.2.1	Runtime Boundary	23
5.2.2	Implementation Areas	23
5.3	Backend and Operator Console	23
5.3.1	Backend Responsibilities	23
5.3.1.1	Run Control	24
5.3.1.2	Trace Assembly	24
5.3.2	Operator Console	24
5.3.2.1	Run Review and Evidence Navigation	24

5.3.2.2	Overview Telemetry	25
5.3.2.3	Agent Testing	25
5.4	Browser Tool Layer	25
5.4.1	Tool Profiles	25
5.4.2	Execution Discipline	27
5.4.2.1	Observe Before Acting	27
5.4.2.2	Verify After Acting	27
5.5	Agents, Prompts, Memory, and Models	27
5.5.1	Prompt Compilation	27
5.5.2	Memory and Model Use	27
5.5.2.1	Memory as Hint	28
5.5.2.2	Model Selection	28
5.6	Persistence and Observability	28
5.6.1	Stored Records	28
5.6.1.1	Evidence Records	28
5.6.1.2	Telemetry Records	28
5.7	Deployment Direction	29
5.7.1	Company Deployment Flow	29
5.7.1.1	agent_tasks Input Queue	29
5.7.1.2	n8n and Puppeteer Container	29
5.7.1.3	agent_results Output Queue	29
5.7.2	Relation to Open Web Catcher	30
5.8	Implementation Summary	30
6	Evaluation and Testing	31
6.1	Evaluation Context	31
6.1.1	Dataset and Snapshot	31
6.1.2	Dashboard Snapshot	31
6.2	Success Metrics	33
6.2.1	Acceptance Rule	33
6.3	Agent Regression Results	34
6.3.1	Prompt and Tool Iterations	34
6.4	Model, Cost, and Cache Behavior	35
6.4.1	Runtime Model Choice	35
6.5	Provider and Evidence Enrichment	36
6.5.1	Infrastructure Context	36
6.6	Tool Evaluation	36
6.6.1	Tool Health and Usefulness	36
6.7	Remaining Limitations	38

6.7.1	Known Failure Sources	38
6.8	Chapter Summary	38
7	Conclusion and Perspectives	39
7.1	General Conclusion	39
7.2	Objectives Achieved	39
7.3	Main Contributions	39
7.4	Technical and Personal Learning	40
7.5	Limitations	41
7.5.1	Runtime Instability	41
7.5.2	Evaluation Scope	41
7.6	Future Work	41
7.6.1	Evaluation and Regression	41
7.6.2	Candidate Ranking and Memory	41
7.6.3	Deployment and Review	41
7.7	Final Remarks	42
A	Evaluation Case Studies	43
A.1	Case Study A: Landing Page With Repeated Event Cards	43
A.2	Case Study B: Hosting Page With Popup Interference	43
A.3	Case Study C: Embedded Player With Delayed Manifest	44
A.4	Case Study D: Tokenized Stream URL	45
A.5	Case Study E: Wrong Page Classification	45
A.6	Case Study F: Provider Enrichment Without Abuse Contact	46
A.7	Case Study Summary	46
A.8	Manual Review Checklist for Case Studies	47
B	Agent Prompt Contracts	48
B.1	Orchestrator Agent Prompt	48
B.2	Classification Agent Prompt	49
B.3	Landing-Page Agent Prompt	50
B.4	Hosting-Page Agent Prompt	51
B.5	Embedded-Page Agent Prompt	52
B.6	Provider and Email Prompt Contract	53
C	Detailed Agent Responsibility Views	55
C.1	Orchestrator	55
C.2	Classification Agent	56
C.3	Landing-Page Agent	56
C.4	Hosting-Page Agent	57

C.5	Embedded-Page Agent	57
C.6	Provider and Email Stage	58
C.7	Responsibility Matrix	58
C.8	Why These Boundaries Matter	59
Bibliography		61

List of Figures

1.1	Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.	4
2.1	Methodology selection logic used for the internship work.	6
2.2	DSRM adopted for the project, with Agile-style short iterations inside each phase.	7
2.3	Internship timeline from evidence needs analysis to evaluated OWC architecture.	8
3.1	Comparison of the three page roles used by the extraction workflow.	11
3.2	Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.	12
3.3	Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH. . . .	12
3.4	Observe–act–verify loop used by browser agents.	13
4.1	Use-case overview of the evidence-collection platform.	15
4.2	Component architecture linking the operator console, backend, agent runtime, MCP tooling, persistence, and external services.	16
4.3	End-to-end agent architecture at workflow scale.	17
4.4	UML class diagram of the agent runtime and result objects.	18
4.5	Persistence model for run evidence, telemetry, and review outputs.	18
5.1	n8n workflow screenshots: full sequential workflow and the four specialist agent workflows.	22
5.2	How n8n limitations shaped the Open Web Catcher implementation.	23
5.3	Operator console overview with run, stream-yield, token, cost, tool, and persistence telemetry.	25
5.4	Current Next.js operator-console screens used for launch, review, telemetry, provider enrichment, and runtime configuration.	26
5.5	Browser tool execution loop through MCP and Puppeteer.	26
5.6	Prompt evolution from one-pass intent prompts to evidence-driven agent loops.	28
5.7	Company deployment flow using agent_tasks for input URLs and agent_results for matched URLs with metadata.	30
6.1	Evaluation protocol used for website-batch regression testing.	31
6.2	Regression-batch improvement across the main browser-agent KPIs.	34
C.1	Orchestrator-agent design card.	55
C.2	Classification-agent design card.	56

C.3 Landing-page-agent design card. 56

C.4 Hosting-page-agent design card. 57

C.5 Embedded-page-agent design card. 58

C.6 Provider-analysis and email-generation design card. 58

List of Tables

2.1	Methodology selection for the evidence-collection project.	6
2.2	DSRM mapping to internship activities.	7
2.3	Functional requirements for the evidence-collection workflow.	9
2.4	Non-functional requirements and design responses.	10
4.1	Main use cases supported by the evidence-collection platform.	16
4.2	Routing decision table replacing the duplicated orchestrator flowchart.	19
4.3	Agent responsibility boundaries.	20
4.4	Persistence groups used by the review workflow.	20
5.1	n8n prototype lessons and implementation responses.	22
5.2	Implementation areas of Open Web Catcher.	24
5.3	Browser tool groups used by the specialist agents.	27
5.4	Observability records used during run review.	29
6.1	Evaluation context for the website regression batch.	32
6.2	Dashboard snapshot used as operational telemetry, separate from regression- batch metrics.	32
6.3	Evaluation metrics used for agent, model, tool, and cost analysis.	33
6.4	Prompt regression history measured on the website batch.	34
6.5	Main browser-agent KPIs after prompt and tool iterations.	35
6.6	Model, token, cost, and cache telemetry used during evaluation.	36
6.7	Provider-enrichment telemetry for extracted stream evidence.	37
6.8	Tool-health telemetry and main tool-regression fixes.	37
7.1	Summary of achieved objectives.	40
A.1	Landing-page regression case: repeated event cards.	43
A.2	Hosting-page regression case: popup and tab focus recovery.	44
A.3	Embedded-player regression case: delayed manifest generation.	44
A.4	Stream-evidence case: tokenized URL preservation.	45
A.5	Routing case: mixed-role page classification.	45
A.6	Provider-enrichment case: useful metadata without abuse contact.	46
A.7	Summary of evaluation case studies and design impact.	46
C.1	Orchestrator responsibility checklist.	55
C.2	Classification-agent responsibility checklist.	56

C.3	Landing-agent responsibility checklist.	57
C.4	Hosting-agent responsibility checklist.	57
C.5	Embedded-agent responsibility checklist.	58
C.6	Provider/email responsibility checklist.	59
C.7	Detailed agent responsibility matrix.	59

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASB	Azure Service Bus
CDN	Content Delivery Network
CDP	Chrome DevTools Protocol
CRISP-DM	Cross Industry Standard Process for Data Mining
DASH	Dynamic Adaptive Streaming over HTTP
DB	Database
DNS	Domain Name System
DOM	Document Object Model
DSRM	Design Science Research Methodology
E2E	End-to-End
HLS	HTTP Live Streaming
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model
M3U8	UTF-8 M3U playlist format
MCP	Model Context Protocol
MPEG-DASH	MPEG Dynamic Adaptive Streaming over HTTP
MPD	Media Presentation Description
OWC	Open Web Catcher
PFE	Projet de Fin d'Etudes
RDAP	Registration Data Access Protocol
RPD	Requests Per Day
RPM	Requests Per Minute
SSE	Server-Sent Events
SQL	Structured Query Language
TPM	Tokens Per Minute
TTL	Time To Live
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience

Acronym	Meaning
WHOIS	Domain registration lookup protocol or service

Terminology

Term	Meaning in this report
Open Web Catcher (OWC)	The internal name of the complete evidence-collection platform built during the internship. It refers to the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, specialist-agent runtime, and review outputs used to catch open-web evidence from browser-visible streaming pages.
Agent	A model-driven runtime component that receives a bounded task, uses approved tools, and returns structured evidence or a failure reason.
Orchestrator	The control component that classifies the starting URL, prepares handoffs, schedules specialist agents, aggregates evidence, and decides the final run status.
Specialist agent	A browser-facing agent with a narrow responsibility, such as classification, landing-page discovery, hosting-page extraction, or embedded-player inspection.
Landing page	A page that lists events, channels, schedules, or match cards and is mainly useful for discovering downstream hosting candidates.
Hosting page	A page that usually owns play buttons, server/source controls, overlays, and redirects before a stream or embedded player becomes visible.
Embedded page	A direct player or iframe page used as a fallback extraction context when the hosting page exposes a player URL.
Browser automation	Programmatic control of a real browser session to inspect rendered pages, click controls, manage frames, capture screenshots, and observe network activity.
Tool call	A browser, provider, memory, or runtime operation requested by an agent and recorded as part of the run trace.
Handoff	A short structured task brief prepared by the orchestrator for the next specialist agent, including target URL, evidence needed, route source, and navigation policy.

Term	Meaning in this report
Trace	The ordered record of runtime events, model calls, tool calls, screenshots, status changes, costs, and final outputs for one investigation.
Observability	The ability to explain what happened during a run after it finishes, including which agent acted, which tools were used, and why a failure occurred.
Stream evidence	A URL, manifest, media request, screenshot, or player state that supports the claim that a target page exposed unauthorized streaming behavior.
Manifest	A playlist or media presentation file, such as HLS or MPEG-DASH, that points a player toward the media segments required for playback.
Iframe	An embedded browser frame that loads a player or third-party page inside the visible hosting page.
Provider enrichment	The step that resolves collected stream hosts or IP addresses into provider, country, network, or abuse-contact information.
Takedown email	A reviewable draft that groups stream evidence, provider details, screenshots, and rights-owner context for later human use.
Regression batch	A repeated set of website tests used to verify that a prompt, model, or tool change fixes new cases without breaking previously working ones.
Evidence artifact	Any persisted output used for review, including screenshots, streams, provider rows, emails, model usage, and tool results.
Runtime event	A timestamped event emitted during execution, such as agent start, tool success, screenshot capture, stream extraction, or failure.
Prompt	The role and task instruction compiled for an agent before it calls the language model.
Memory hint	A reusable site or layout note injected as soft guidance; it can help the agent but must be confirmed against live browser evidence.
Input tokens	Prompt, trace, tool-output, memory, and context text sent to the model during an LLM call.
Cached input tokens	Input tokens reused through the prompt or context cache instead of being billed and processed as fully new context.

Term	Meaning in this report
Output tokens	Tokens generated by the model as an answer, tool decision, explanation, or structured result.
Total tokens	The aggregate token count used by the dashboard to summarize input and output usage for model calls.
Context window	The maximum amount of prompt, trace, tool output, and memory text that the model can consider in one call.
Prompt cache	A cache mechanism used to reuse repeated prompt and context content across similar model calls.
Cache hit rate	The share of model input served from cache within a specific dashboard or model-usage view. It must be read with its denominator.
LLM call	One recorded invocation of the language model, including the prompt, model name, token counters, and cost estimate.
Model spend	The recorded or estimated monetary cost of model calls for a run, batch, or dashboard snapshot.
Cost per run	Model spend normalized by the number of runs in the selected view; it is used to compare evaluation versions.
Tool success rate	Successful observed tool calls divided by observed tool calls in the same telemetry snapshot.
Stream yield	The share of runs that produced at least one stream, manifest, media, or embedded-player evidence candidate.
Strict stream success	A stricter success measure where a run counts only when it reaches reviewable stream evidence, not merely a completed status.
Pipeline run	One end-to-end investigation from a seed URL through classification, specialist routing, evidence collection, enrichment, and final status.
Active run	A pipeline run that is currently queued, running, or waiting on browser/model work.
Queued run	A run or task that has been accepted but has not yet started browser execution.
Run status	The final or current state of a pipeline run, such as success, partial, failed, cancelled, no_streams, or no_hosting_pages.
no_streams	A terminal status used when the browser path was explored but no reviewable media, manifest, stream, or embedded evidence was found.
no_hosting_pages	A terminal status used when a landing-page investigation did not discover credible downstream hosting candidates.

Term	Meaning in this report
Evidence bundle	The grouped review output for one run, including streams, screenshots, provider rows, email drafts, status, costs, and trace links.
Provider coverage	The extent to which collected stream or host evidence could be enriched with provider, network, or registration context.
Dashboard snapshot	A live operator-console aggregate at a specific time. It should not be mixed blindly with controlled regression-batch metrics.
Persistence health	A dashboard view showing whether runs, agent runs, runtime events, model calls, tool calls, and outputs are being stored correctly.
Tokenized URL	A media or manifest URL containing temporary query parameters, signatures, or session-bound path values.
Queue message	A structured payload sent through a queue to start work or return results without blocking a synchronous web request.
agent_tasks	The Azure Service Bus input queue used in the company deployment flow to receive target URLs and task metadata.
agent_results	The Azure Service Bus output queue used in the company deployment flow to return matched URLs and metadata.
Browser profile	A browser context with its own state, cookies, cache, and execution settings for a run or agent.
Profile-scoped session	A browser session tied to one profile so page state can be preserved while avoiding uncontrolled sharing across unrelated runs.

Introduction

The internship was carried out in a practical engineering context: Soft Stars needed a clearer way to inspect suspected illegal sports-streaming websites and preserve the evidence collected during the browser investigation. The project was connected to the beIN SPORTS rights-enforcement context, where a reviewer does not only need a final URL. The reviewer needs to understand which page was visited, which player path was followed, which stream or embedded URL was found, and which screenshots or traces support the result.

1.1 Host Organization and Internship Context

1.1.1 Host Organization

Soft Stars is a software company working on digital media intelligence and technical support for rights-enforcement operations. Within this environment, the expected PFE result was not a purely theoretical study. It had to be a practical contribution that could be reviewed by engineers, discussed with the company team, and continued after the internship.

1.1.2 Operational Context

Sports-streaming investigations are difficult because the useful evidence is often hidden behind dynamic pages, scripts, popups, server buttons, redirects, and nested iframes. A suspicious website can expose one page for discovery, another page for hosting the visible player, and a separate embedded player URL that finally produces media requests. The evidence path therefore matters as much as the final stream candidate.

In this setting, a useful run must explain what was observed and why the system accepted or rejected the result. The project therefore became a software engineering problem around browser automation, page classification, evidence capture, observability, and repeatable evaluation.

1.2 Problem Statement

Illegal sports streaming websites are difficult to analyze with static scripts. Their useful evidence can appear only after the browser renders JavaScript, opens a tab, closes a popup, clicks a play overlay, switches a server, or waits for a media manifest to appear in the network activity. A simple HTTP request can miss the real player state, while manual review becomes slow and inconsistent when the number of targets increases.

The problem addressed in this report is the following: how can a browser-based system collect reviewable evidence from dynamic illegal streaming websites while keeping the investigation traceable, repeatable, and measurable? The answer explored during the internship is a

multi-agent workflow where each agent owns a bounded part of the browsing task and where screenshots, network signals, model calls, tool calls, and final outputs are stored for review.

1.3 Internship Mission and Scope

1.3.1 Assigned Mission

My mission during the internship was to study the evidence-collection workflow, validate whether an LLM-guided browser agent could operate on real streaming sites, and design Open Web Catcher (OWC) around that idea. The work started with an n8n prototype because it allowed fast experimentation on real targets. That prototype was useful for testing browser-agent behavior, but it also exposed limits around traceability, memory, evaluation, browser state, and long-running executions.

The second part of the internship transformed those lessons into Open Web Catcher, the internal name used for the maintained evidence-collection platform. The name was chosen because the system catches open-web evidence from browser-visible streaming pages: page roles, screenshots, stream candidates, provider metadata, traces, and review outputs. OWC is the whole project boundary, not only the backend. It groups the FastAPI control API, the Next.js operator console, PostgreSQL persistence, MCP browser tools, specialist agents, and provider/email review stages.

The goal was not to claim that every company operation had been replaced. The goal was to produce a stronger engineering foundation: specialized agents, clearer handoffs, profile-scoped browser tools, persistent traces, model and tool telemetry, screenshot storage, provider enrichment, and regression-oriented evaluation.

1.3.2 Mission Deliverables

The internship produced three connected deliverables. The first was an exploratory n8n workflow used to validate the browser-agent idea on real pages. The second was OWC, a maintained platform that separated page classification, landing exploration, hosting interaction, embedded-player inspection, provider enrichment, and email drafting. The third was an evaluation approach based on regression batches, model and tool telemetry, provider coverage, and operator review.

1.3.3 Scope Boundaries

The work remained focused on evidence collection and review support. It did not send take-down emails automatically, it did not store the client's enterprise production database, and it did not attempt to solve every possible anti-bot or availability issue on hostile websites. The final result should therefore be read as a foundation for a rights-enforcement workflow, not as a finished legal enforcement platform.

1.4 Objectives and Main Contributions

1.4.1 Objectives

The project objectives were:

- analyze how illegal streaming pages expose evidence through landing, hosting, and embedded-player flows;
- validate browser-agent navigation on real dynamic targets using an exploratory n8n workflow;
- design the maintainable Open Web Catcher platform with specialized agents, explicit hand-offs, and review-oriented persistence;
- preserve evidence through screenshots, stream candidates, provider enrichment, runtime events, model usage, and browser tool traces;
- evaluate the workflow with repeatable website batches instead of relying on isolated successful demonstrations.

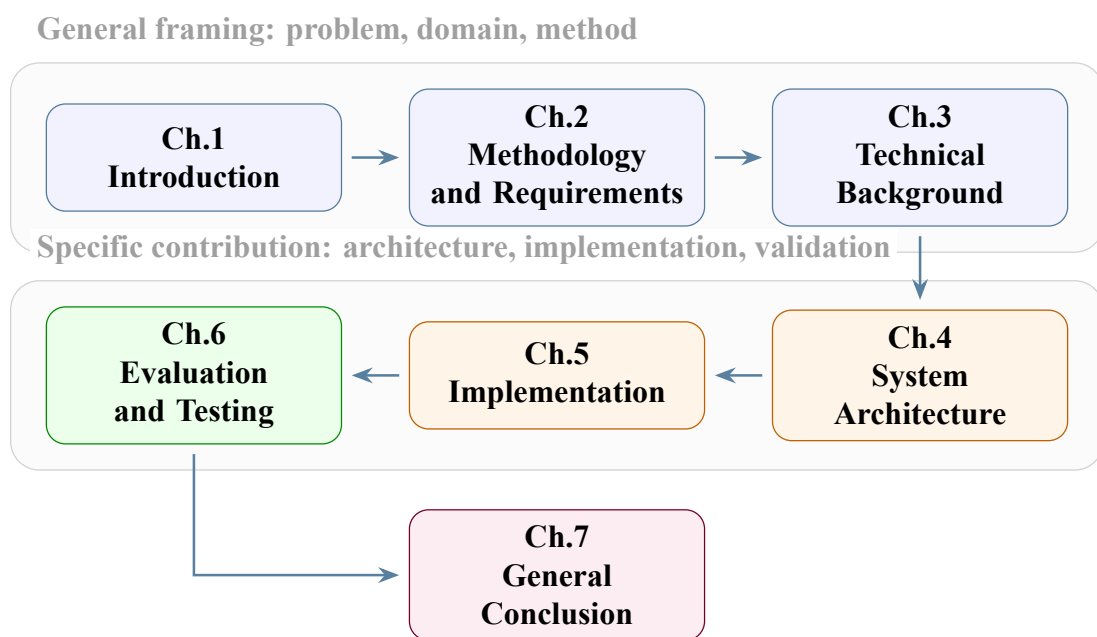
1.4.2 Main Contributions

The main contributions of the internship are:

- a practical framing of sports-streaming evidence collection as a browser automation and observability problem;
- an n8n prototype that validated the core browser-agent approach and revealed its operational limits;
- a staged multi-agent architecture separating classification, landing-page discovery, hosting-page interaction, embedded-player inspection, provider analysis, and email drafting;
- a persistence model that keeps both final evidence and the failure path of each run;
- an evaluation approach based on regression batches, tool telemetry, model cost, cache behavior, and provider coverage.

1.5 Report Organization

Chapter 2 explains the methodology and the requirements that shaped the work. Chapter 3 introduces the technical concepts needed to understand dynamic streaming pages, media evidence, browser automation, and agentic tool use. Chapter 4 presents the architecture of the reference system. Chapter 5 describes the implementation path from the n8n prototype to OWC and the queue-based deployment direction. Chapter 6 evaluates the workflow through website batches, dashboard telemetry, model metrics, and tool behavior. Chapter 7 concludes with the achieved objectives, limits, and future work.



The report connects the internship problem, design choices, implementation, evaluation, and final perspectives.

Figure 1.1: Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.

Methodology, Requirements, and Design Constraints

The internship combined domain analysis, fast prototyping, software design, and evaluation on unstable websites. A normal linear development process would not have been enough, because many problems appeared only after running the browser against real pages. The methodology therefore had to support both structured artifact design and short feedback cycles.

2.1 Methodological Choices

Three methodological references were considered. Agile practices matched the daily work rhythm: inspect a failed run, change a prompt or browser tool, test again, and preserve the lesson. CRISP-DM was useful as a secondary lens because the project involved business understanding, data observation, evaluation, and operational review. However, the project was not mainly a data-mining or model training project. Design Science Research Methodology (DSRM) was the best primary frame because the internship produced an artifact intended to solve an organizational problem.

2.1.1 Selection Logic

The chosen method had to explain three kinds of work at the same time: an engineering artifact, a changing set of target websites, and repeated correction cycles after real browser runs.

2.1.1.1 Artifact Fit

The project had to produce a concrete workflow, not only an analysis. DSRM therefore provided the main structure because it links a problem, an artifact, a demonstration, and an evaluation.

2.1.1.2 Iteration Fit

Agile practices were kept as the day-to-day discipline. They made sense for prompt fixes, tool changes, and browser-action corrections because each run could reveal a new obstacle.

2.1.1.3 Evaluation Fit

CRISP-DM was kept as a supporting lens for understanding the domain, looking at collected run data, and interpreting regression batches. It was not used as the main method because the project was not a predictive modeling project.

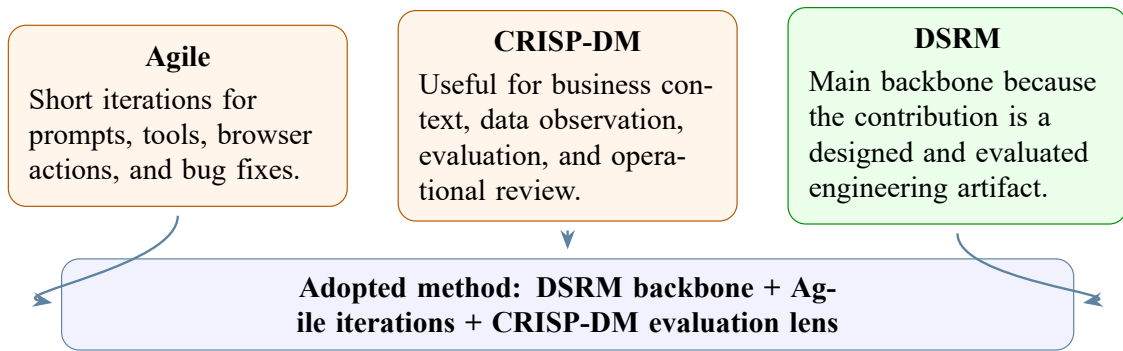


Figure 2.1: Methodology selection logic used for the internship work.

Method	Useful contribution	Limit in this project
Agile	Short iterations for prompt, tool, and browser behavior corrections.	Does not by itself justify the artifact or its evaluation.
CRISP-DM	Clear thinking about business context, data, evaluation, and operational review.	Too centered on datasets and predictive modeling for this artifact-driven work.
DSRM	Connects problem identification, artifact design, demonstration, evaluation, and communication.	Needs fast engineering iterations inside each phase.

Table 2.1: Methodology selection for the evidence-collection project.

2.2 Adopted Methodology

The report uses DSRM as the backbone and Agile-style iteration as the execution discipline. In practice, this meant that each failed browser run could send the work back to a prompt rule, tool description, memory policy, routing decision, or evaluation criterion. This was necessary because illegal streaming pages are not stable test fixtures: layouts change, popups appear, server buttons move, and media URLs are often generated only after user interaction.

2.2.1 Backbone and Supporting Lenses

2.2.1.1 DSRM Backbone

The chapter sequence follows the same logic as the project work: identify the problem, define objectives, build the artifact, demonstrate it on real cases, evaluate it, and communicate the result.

2.2.1.2 Agile Iterations

Inside each DSRM phase, the implementation used short iterations. A failed click, missing iframe, or premature harvest became a small correction task rather than a reason to restart the whole design.

2.2.1.3 CRISP-DM Evaluation Lens

CRISP-DM helped structure how batch results, tool telemetry, provider rows, and model costs were interpreted. It was mainly useful once the prototype started producing repeatable run data.

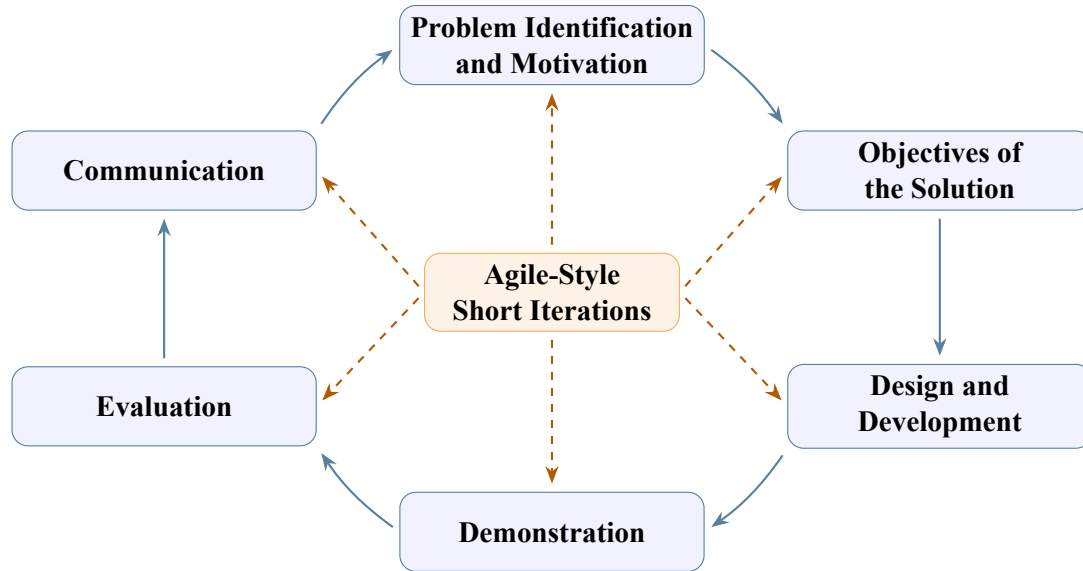


Figure 2.2: DSRM adopted for the project, with Agile-style short iterations inside each phase.

DSRM phase	Application during the internship	Main output
Problem identification	Study the rights-enforcement need and the practical limits of manual review and static scraping.	Evidence-collection problem statement.
Objectives	Define the need for page classification, browser interaction, screenshots, stream evidence, provider enrichment, and traceability.	Functional and non-functional requirements.
Design and development	Build an n8n prototype, then derive the Open Web Catcher platform with specialized agents and persistence.	Prototype and OWC architecture.
Demonstration	Run the workflow on representative landing, hosting, embedded, popup, iframe, and tokenized-player cases.	Demonstration traces and screenshots.
Evaluation	Compare prompt, model, tool, and batch behavior with acceptance rules and regression checks.	Evaluation metrics and failure analysis.
Communication	Produce the final PFE report and diagrams.	Submitted report.

Table 2.2: DSRM mapping to internship activities.

2.3 Project Evolution

The work evolved in eight practical phases: domain and evidence needs analysis, streaming-site behavior study, n8n prototype creation, single-agent validation, OWC architecture design, multi-agent routing and tool separation, evaluation batches, and final report synthesis. Figure 2.3 summarizes this progression.

2.3.1 Main Phases

2.3.1.1 Exploration and Prototype

The first phases focused on understanding what evidence was expected, how streaming sites expose player paths, and whether n8n could coordinate enough browser actions to validate the idea.

2.3.1.2 OWC Architecture and Evaluation

The later phases focused on separating agents, defining handoffs, storing traces, and testing the workflow against regression batches instead of isolated successful examples.

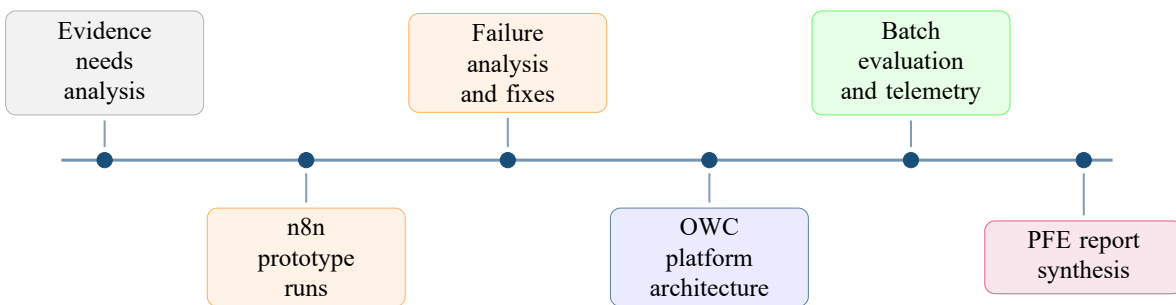


Figure 2.3: Internship timeline from evidence needs analysis to evaluated OWC architecture.

The n8n prototype and OWC should not be read as competing products. The prototype answered whether the approach was feasible. OWC answered what the complete system would need to become maintainable, observable, and measurable.

2.4 Functional Requirements

The functional requirements were derived from the evidence workflow itself. A successful investigation must move from a seed URL to a reviewable evidence bundle.

2.4.1 Requirement Groups

2.4.1.1 Ingestion and Routing

The first group covers URL ingestion, page classification, specialist routing, and trace creation.

2.4.1.2 Evidence Collection

The second group covers landing candidates, hosting interaction, embedded-player inspection, screenshots, streams, and provider enrichment.

ID	Requirement	Priority
FR-1	Ingest a target URL and create a traceable investigation run.	Critical
FR-2	Classify the page as landing, hosting, embedded, other, or blocked.	Critical
FR-3	Route work to the correct specialist agent.	Critical
FR-4	Extract landing-page match cards, tabs, event links, and host candidates.	High
FR-5	Operate hosting-page server controls, overlays, and play buttons.	Critical
FR-6	Inspect embedded players and recover stream or manifest evidence.	Critical
FR-7	Capture screenshots at meaningful evidence or failure moments.	High
FR-8	Persist stream, screenshot, provider, model, tool, and event records.	High
FR-9	Enrich collected streams with provider and registration metadata.	Medium
FR-10	Provide an operator review surface for traces and evidence.	High

Table 2.3: Functional requirements for the evidence-collection workflow.

2.5 Non-Functional Requirements

The non-functional requirements were just as important as extraction accuracy. The system had to make a failure explainable, not merely return a final “failed” label.

2.5.1 Quality Attributes

2.5.1.1 Traceability and Observability

Each run had to preserve enough events, screenshots, model calls, and tool calls to explain how the result was reached.

2.5.1.2 Maintainability and Cost Control

The implementation had to separate prompts, browser tools, agents, and persistence while tracking model usage across regression batches.

2.6 Chapter Summary

The methodology was chosen because the internship produced an engineering artifact under uncertain runtime conditions. DSRM provided the overall research logic, while short iterations made it possible to react to real browser failures. The next chapter defines the technical concepts used by the artifact: streaming-site page roles, browser-rendered evidence, media manifests, browser automation, and agentic tool use.

ID	Requirement	Design response
NFR-1	Traceability	Store runtime events, screenshots, tool calls, model calls, and final evidence under the same run identifier.
NFR-2	Maintainability	Split prompts, browser tools, agents, persistence, and UI review into explicit components.
NFR-3	Reliability	Use bounded tool budgets, page-type-specific tools, popup handling, and fail-closed routing.
NFR-4	Observability	Keep normalized records for agent runs, tool calls, costs, prompt compilations, and errors.
NFR-5	Cost control	Track token usage, cached input, model costs, and average cost per working website.
NFR-6	Scalability	Support dataset-backed batches and worker-style execution.
NFR-7	Portability	Keep runtime settings configurable and browser services separable from the API.

Table 2.4: Non-functional requirements and design responses.

Technical Background

Before describing the architecture, it is useful to clarify why illegal streaming pages are hard to inspect and why the project used browser agents instead of static scraping. The concepts below are limited to what is needed for the implementation and evaluation chapters.

3.1 Streaming Website Structure

The websites targeted by the project usually separate discovery pages from playback pages. A landing page lists events, channels, schedules, or match cards. A hosting page contains the visible player shell, server buttons, play overlays, and popup traps. An embedded page is a direct player or iframe URL that may expose the final media requests. Separating these roles is important because each page type requires different actions and evidence checks.

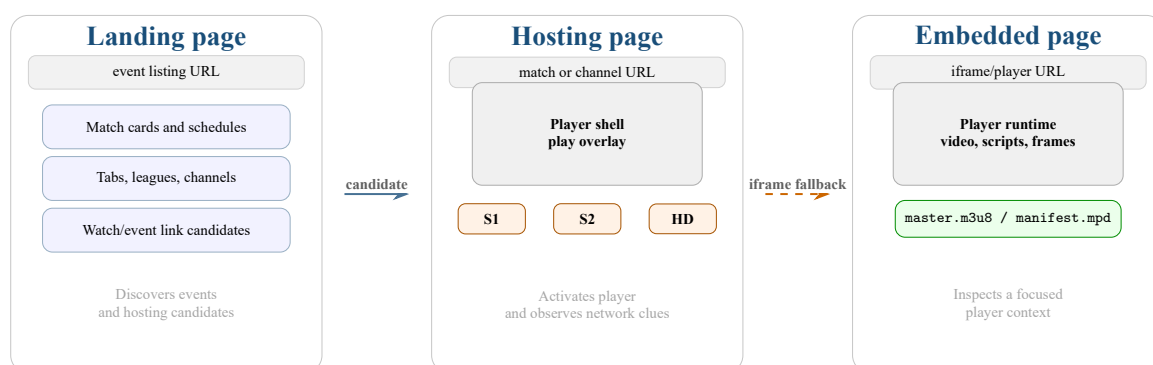


Figure 3.1: Comparison of the three page roles used by the extraction workflow.

Sports piracy monitoring is an evidence problem as much as a discovery problem. Rights holders need to know what content was exposed, which page exposed it, what technical indicators were visible, and whether the result can support a later enforcement review. In practice, this means the system must keep the path from a suspicious page to the player or stream candidate, because illegal services frequently change domains, mirrors, and delivery infrastructure.

3.2 Browser-Rendered Pages and the DOM

Modern streaming pages are not fully represented by their initial HTML. The browser builds a Document Object Model (DOM), runs JavaScript, loads iframes, changes visible state after clicks, and emits network requests during playback. For this project, the DOM matters because it exposes visible links, buttons, frames, match cards, server controls, and player containers that a static HTTP response can miss [8].

A browser automation layer can inspect this rendered state and interact with it. That makes it possible to click tabs, close popups, activate a player, switch servers, capture screenshots, and listen to network traffic. Chrome DevTools Protocol (CDP), Puppeteer, and Playwright expose this kind of browser control and instrumentation, which is why they are relevant to evidence collection instead of ordinary page scraping [7, 5, 6].

3.3 Media Streaming and Manifests

The most useful stream evidence is often not a complete video file. It is usually a manifest or media request generated after the player is activated. In HTTP Live Streaming (HLS), a player can load an .m3u8 playlist that points to media segments and alternative quality levels. In MPEG-DASH, the player can load a media presentation description that gives similar adaptive playback information [17, 18].

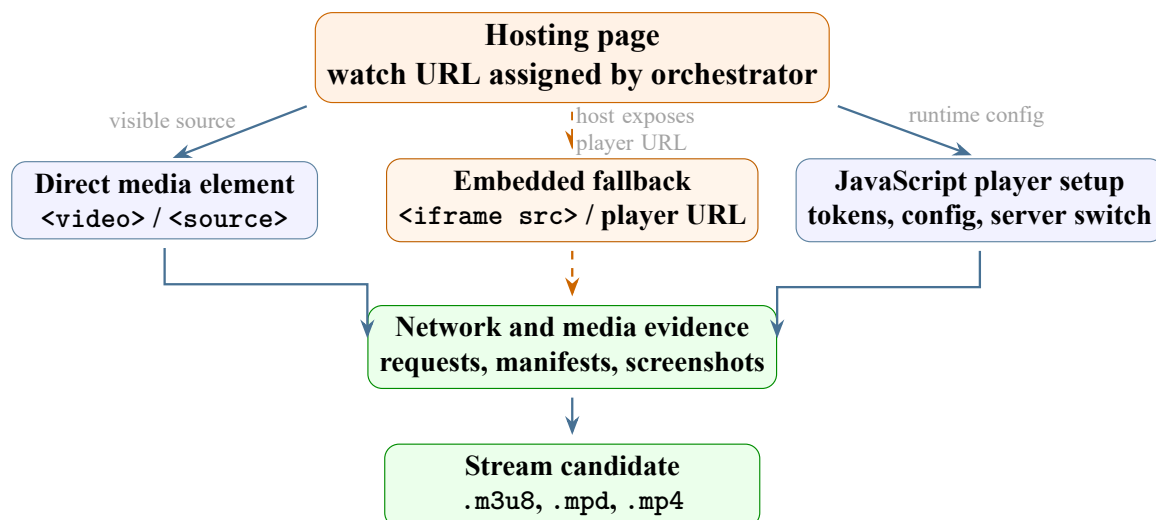


Figure 3.2: Main paths from a hosting page to media evidence, with embedded pages treated as an explicit fallback when the hosting page exposes a player URL.

Manifests are valuable because they show that the page resolved toward playable media infrastructure. They can also expose hostnames, CDN paths, tokens, quality variants, and referer-sensitive URLs. The goal of the system is not to download protected content. It is to capture enough technical evidence to show the path from a suspicious page to a stream candidate.

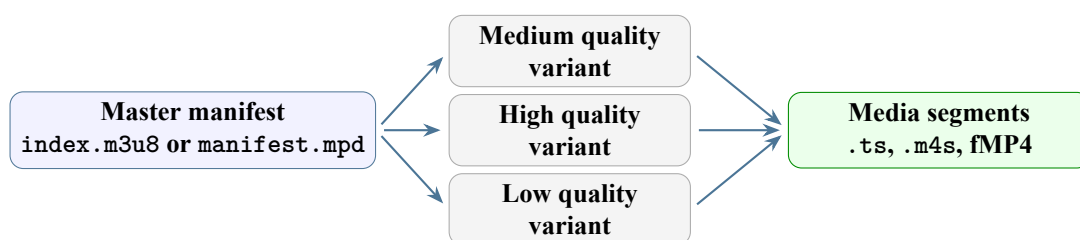


Figure 3.3: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

Some stream URLs are tokenized. In this report, a tokenized URL is a media or manifest URL that contains temporary query parameters or signed path segments. It may work only for a short period, for one browser session, or with a specific referer. The system therefore stores tokenized URLs as evidence artifacts rather than treating them as permanent public links.

3.4 Runtime Obstacles

Illegal streaming pages are deliberately noisy. They often contain advertisement overlays, popup windows, new-tab traps, blocked requests, Cloudflare-like challenge pages, repeated server buttons, and redirects that can move the browser away from the useful player. These obstacles make one-pass scraping fragile and explain why the system records screenshots, network evidence, and failure reasons instead of relying only on final text output [16, 19].

The practical consequence is that the agent must verify state after each important action. A click is not successful because the tool returned without an exception; it is successful only if the visible page, frame tree, or network traffic changed in a useful way. This is why the implementation distinguishes navigation tools, interaction tools, inspection tools, screenshot tools, and harvest tools.

3.5 Agentic Workflows and Tool Use

The project uses the agent idea in a narrow engineering sense. An agent receives a task, observes the current browser state, chooses an allowed tool, reads the tool result, and decides whether more action is needed. The implementation keeps this loop constrained by the page role so that landing, hosting, and embedded work do not drift into each other.

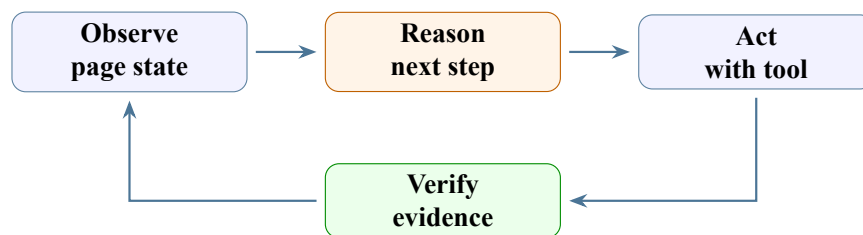


Figure 3.4: Observe–act–verify loop used by browser agents.

Tool use is useful only when the tool contract is clear. The model should know what a tool can do, what evidence it returns, and when it should be used. The Model Context Protocol (MCP) gives a practical way to expose external tools to agents through explicit input and output contracts [11].

The context window is the amount of prompt, trace, tool output, and memory text that the model can consider in one call. Cached input tokens are parts of that context reused through a prompt cache, while output tokens are generated by the model. These counters matter because browser runs can produce long traces and because repeated evaluation batches become expensive without cache visibility [4, 3].

3.6 Evidence Traceability

The final concept is traceability. A run should not only say that a site worked or failed. It should preserve the route, page type, screenshots, stream candidates, provider enrichment, model calls, browser tool calls, and failure mode. This trace is what allows an operator or engineer to decide whether a result is trustworthy, whether a prompt change caused a regression, or whether the website itself blocked playback.

These technical ideas lead directly to the architecture in the next chapter. The system separates page roles, gives each role a specialist agent, restricts the tool surface, and stores evidence as normalized records so that both successful and failed investigations can be reviewed.

System Architecture

The Open Web Catcher (OWC) architecture was designed around one constraint: every result must be reviewable. A final stream URL is useful only if the system can also explain how the browser reached it, which agent made the decision, which tool was used, which screenshot supports the claim, and where the workflow failed when no stream was found.

4.1 Use-Case View

OWC has two main human roles. The operator launches investigations and reviews evidence. The engineer maintains runtime settings, models, tools, datasets, and evaluation behavior. The system itself turns a target URL into a traceable investigation run. Keeping these roles separate matters because an operator needs a readable evidence bundle, while an engineer needs enough runtime detail to diagnose prompt, tool, model, or persistence failures.

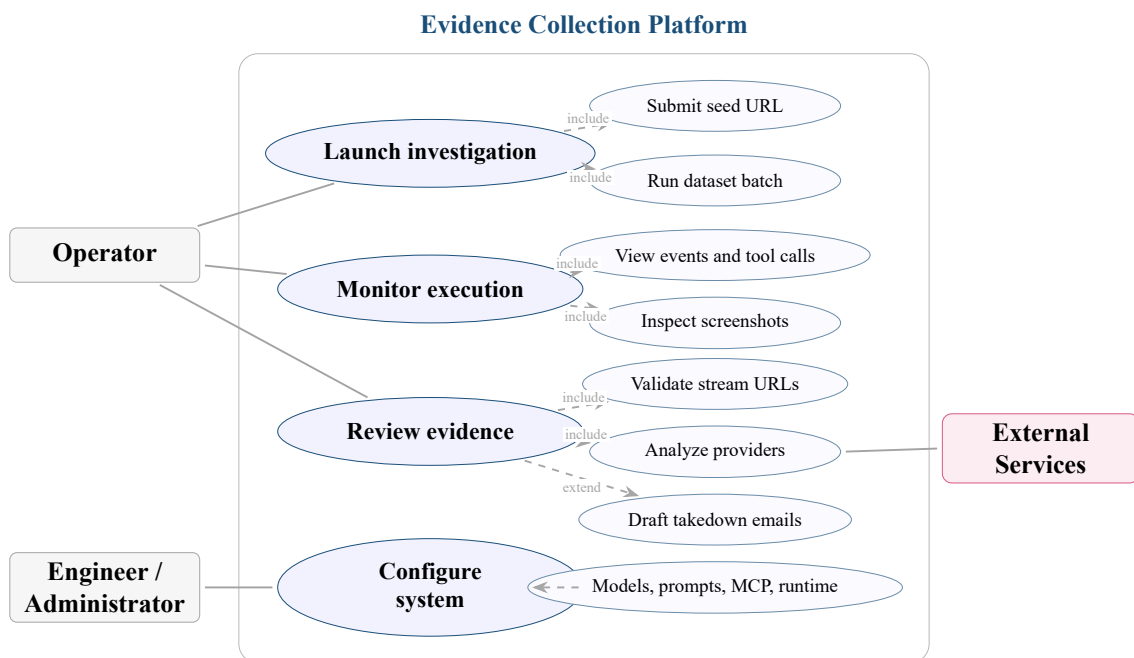


Figure 4.1: Use-case overview of the evidence-collection platform.

4.2 Component Architecture

OWC is split into the Next.js operator console, FastAPI backend, agent runtime, MCP browser tools, PostgreSQL persistence, and external services. The console does not own business logic or database access. FastAPI starts jobs, resolves settings, assembles traces, and

Use-case group	User intent	System response
Launch investigation	Submit one URL or a dataset batch.	Create a run, validate runtime readiness, and start execution.
Monitor execution	Follow agent transitions, browser actions, screenshots, and errors.	Stream active state and persist runtime events.
Review evidence	Inspect streams, screenshots, provider results, costs, and draft emails.	Assemble a reviewable evidence bundle.
Tune runtime	Adjust models, prompts, browser settings, tool profiles, and pricing.	Store configuration and expose preflight feedback.

Table 4.1: Main use cases supported by the evidence-collection platform.

reads or writes normalized rows. Browser interaction is handled through MCP tools so that the agent runtime is not tied to one uncontrolled browser script.

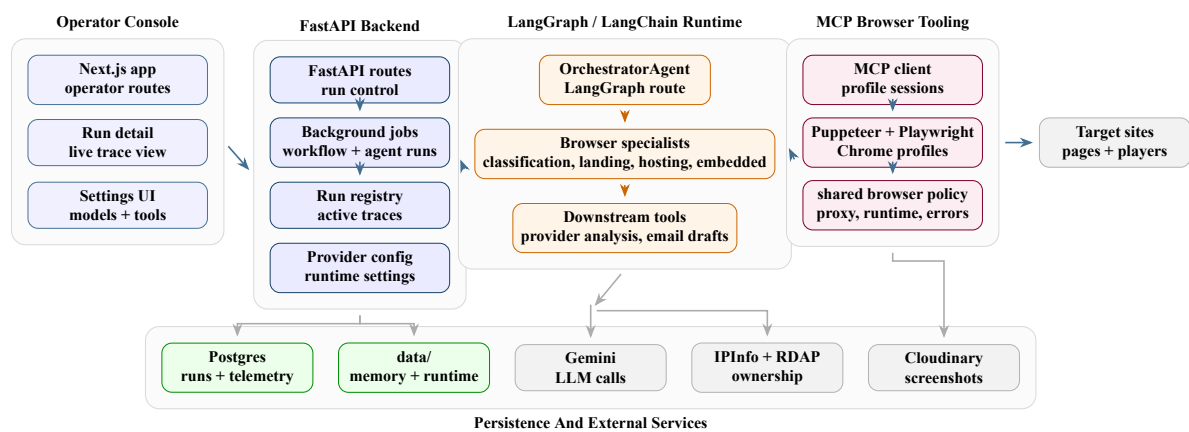


Figure 4.2: Component architecture linking the operator console, backend, agent runtime, MCP tooling, persistence, and external services.

This separation makes failure diagnosis practical. A failed run can be traced to a route decision, browser action, model response, tool timeout, provider lookup, or persistence issue. Without these boundaries, every error would appear as the same generic extraction failure.

4.3 Agent Pipeline

The pipeline is staged. The orchestrator receives the seed URL and controls the run. It calls the classification agent first, then routes to landing, hosting, or embedded specialists. After stream evidence exists, provider analysis and email drafting can run. If no stream evidence exists, those downstream stages are skipped. Figure 4.3 shows the normal flow and the lower auxiliary lane used for skipped or downstream work.

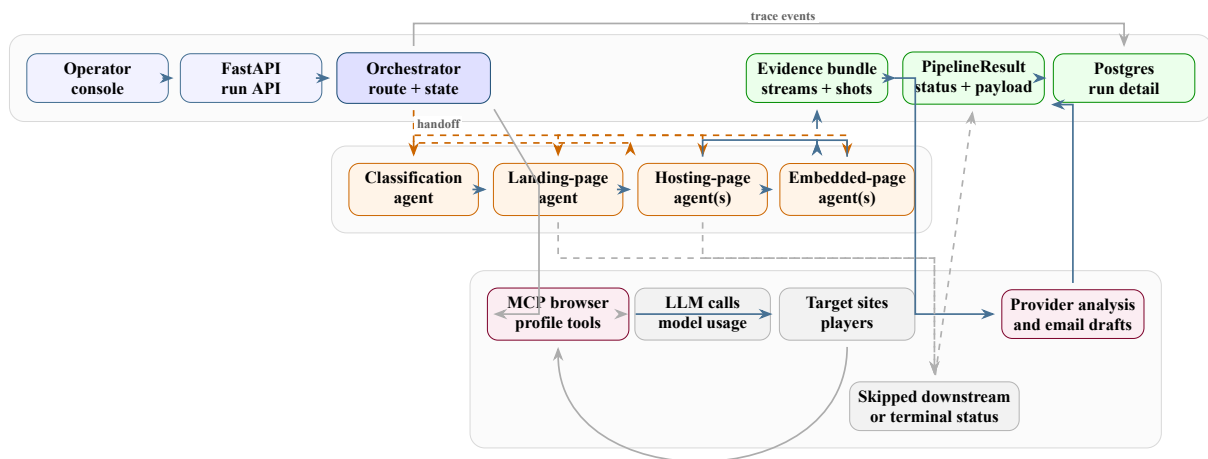


Figure 4.3: End-to-end agent architecture at workflow scale.

4.4 Routing and Handoff Rules

The orchestrator is the only component allowed to route between agents. Agents do not call each other directly. Each specialist receives a handoff containing the target URL, page type, evidence checklist, route source, known candidates, memory hints, and navigation policy. This keeps the task local and prevents a landing agent from drifting into provider analysis or an embedded agent from crawling a full website.

4.5 Agent Runtime Class View

The class view summarizes the agent runtime without showing every implementation detail. The ownership boundary matters: the orchestrator owns the pipeline state and handoff creation, while the specialists share the same model/tool loop shape and return typed results.

The class view is also the basis for the external UML diagram that can be redrawn and exported separately. The report keeps the diagram content focused on runtime responsibilities and result ownership.

4.6 Data and Evidence Persistence

The persistence model stores both the final answer and the path that produced it. A final JSON payload is not enough for this project because the operator needs to inspect screenshots, streams, provider rows, model usage, tool calls, prompt compilations, runtime events, and failure reasons separately.

4.7 Chapter Summary

OWC is a controlled evidence pipeline rather than one unrestricted browser agent. The operator starts a run, FastAPI controls execution and persistence, the orchestrator routes work, specialists collect browser evidence, and normalized records preserve both successful outputs

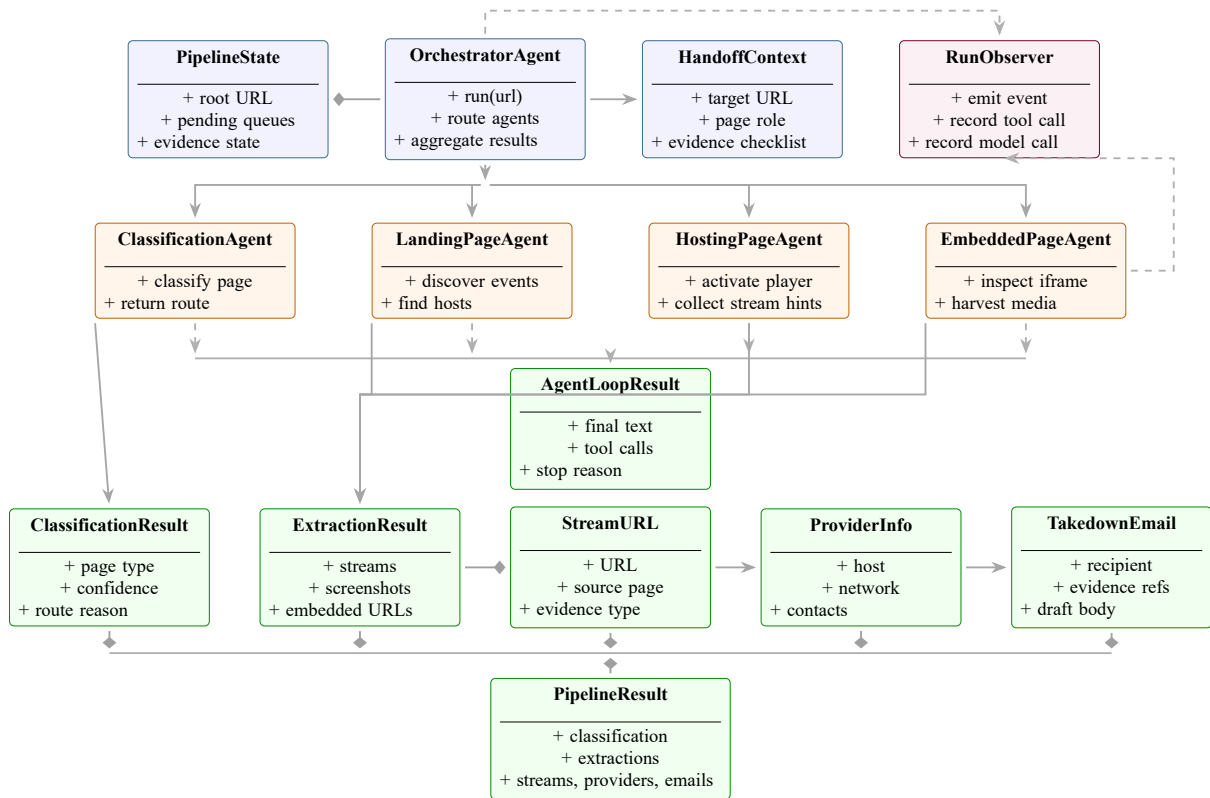


Figure 4.4: UML class diagram of the agent runtime and result objects.

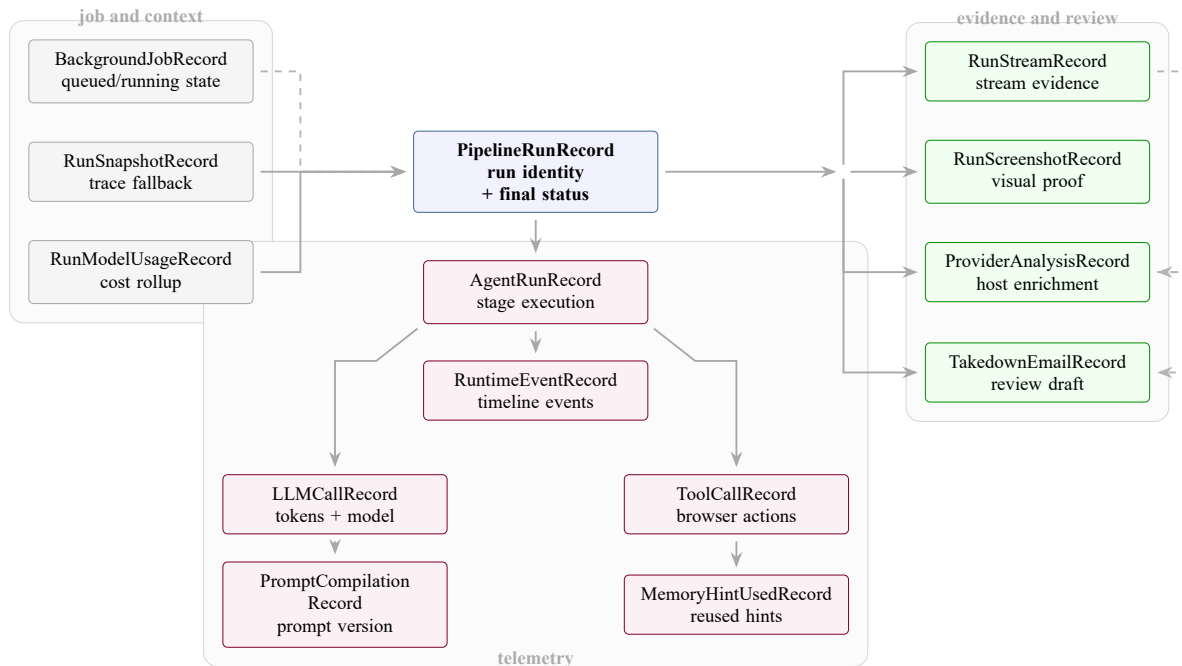


Figure 4.5: Persistence model for run evidence, telemetry, and review outputs.

Detected page type	Routed agent	Expected input	Expected output	Skip or stop condition
Landing page	Landing-page agent	Seed URL, visible listings, event cards, and navigation policy.	Hosting candidates, event links, screenshots, and route notes.	Stop as <code>no_hosting_pages</code> when no credible host candidate is found.
Hosting page	Hosting-page agent	Watch page, server buttons, play overlays, popup policy, and known candidates.	Streams, screenshots, embedded URLs, and player diagnostics.	Stop as <code>no_streams</code> when activation attempts do not create media evidence.
Embedded page	Embedded-page agent	Iframe or player URL produced by routing or hosting inspection.	Manifest/media candidates, screenshots, and frame diagnostics.	Return failure reason when the frame is blocked or no media is observable.
Blocked or irrelevant page	No browser specialist beyond classification.	Current URL, screenshot, and classification reason.	Terminal status and failure evidence.	Skip extraction, provider analysis, and email drafting.
Stream evidence exists	Provider and email stages	Stream URL, source page, screenshot references, and rights context.	Provider rows and reviewable email drafts.	Skip when no stream or embedded evidence was collected.

Table 4.2: Routing decision table replacing the duplicated orchestrator flowchart.

and failure paths. The next chapter explains how this design was implemented and how the n8n prototype influenced OWC.

Stage	Responsibility	Output
Orchestrator	Build the route, prepare handoffs, aggregate results, and decide final status.	PipelineResult and trace events.
Classification	Identify whether the current page is landing, hosting, embedded, blocked, or irrelevant.	ClassificationResult.
Landing	Discover event cards, watch links, hosting candidates, and visible player hints.	ExtractionResult with candidates.
Hosting	Operate server controls, activate playback, handle popups, and capture stream or embedded handoffs.	Streams, screenshots, diagnostics, or embedded URLs.
Embedded	Inspect iframe/player URLs and harvest final media evidence.	Streams, screenshots, and frame diagnostics.
Provider and email tools	Enrich stream hosts and prepare reviewable takedown drafts.	Provider rows and email drafts.

Table 4.3: Agent responsibility boundaries.

Data family	Examples	Why it is stored separately
Run identity	Run ID, root URL, final status, counts, cost totals.	Gives the operator a stable investigation record.
Agent observability	Agent runs, runtime events, model calls, tool calls.	Explains how the result was reached.
Evidence artifacts	Streams, screenshots, embedded URLs, provider rows.	Preserves reviewable evidence.
Review outputs	Decisions, tasks, email drafts, provider contacts.	Supports human review after automated extraction.

Table 4.4: Persistence groups used by the review workflow.

Implementation

The implementation was built in two connected phases. The first phase used n8n to validate the idea quickly on real streaming websites. The second phase turned the lessons from that prototype into Open Web Catcher (OWC), the complete platform boundary for the FastAPI backend, Next.js operator console, PostgreSQL persistence, MCP browser tooling, specialist agents, and review outputs. The deployment direction then focused on the part that could realistically be operationalized during the internship.

5.1 Prototype Phase with n8n

5.1.1 Prototype Goal

The first objective was speed. The prototype had to prove whether a browser-guided workflow could reach useful evidence before investing in a stricter software architecture.

n8n was useful at the start because it allowed rapid experiments with browser steps and agent-like workflows [12]. At this stage, the goal was not to produce the final software architecture. The goal was to test whether an LLM-driven browser workflow could classify streaming pages, navigate through landing and hosting pages, activate players, and return useful evidence.

5.1.2 Prototype Findings

5.1.2.1 What Worked

The prototype showed that an LLM-guided browser workflow could inspect real pages, click through player controls, and return evidence candidates when the page state was preserved long enough.

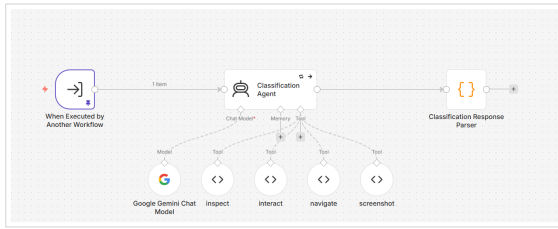
5.1.2.2 What Failed

The same prototype also showed that long browser workflows need stronger state, clearer traces, and repeatable evaluation. Without those pieces, a successful run was hard to compare with a failed one.

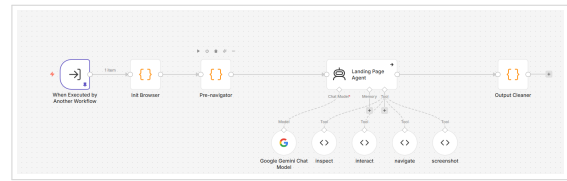
The prototype confirmed that the approach was feasible, but it also showed why a stronger implementation was needed. Browser state was hard to preserve across isolated workflow steps, popup handling was fragile, evidence review was mostly manual, memory was not durable, and prompt changes were difficult to compare without a regression dataset.



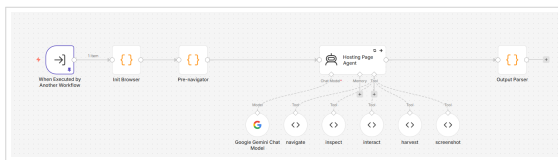
Full n8n workflow



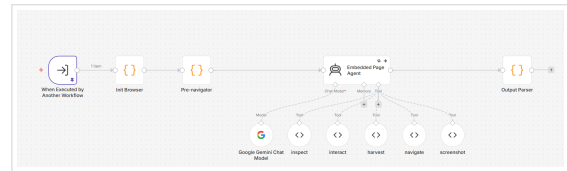
Classification agent



Landing-page agent



Hosting-page agent



Embedded-player agent

Figure 5.1: n8n workflow screenshots: full sequential workflow and the four specialist agent workflows.

Prototype finding	Practical effect	Design response
Sequential workflow	Runs became long and hard to inspect when many hosting targets existed.	Use an orchestrator with explicit queues and stage status.
Weak browser continuity	Browser context could be lost between steps.	Use profile-scoped browser sessions through MCP tools.
Manual review	Failures required manual reconstruction from workflow output.	Persist runtime events, screenshots, model calls, and tool calls.
No batch regression	Prompt changes were judged from small examples.	Build dataset-backed batch evaluation.

Table 5.1: n8n prototype lessons and implementation responses.

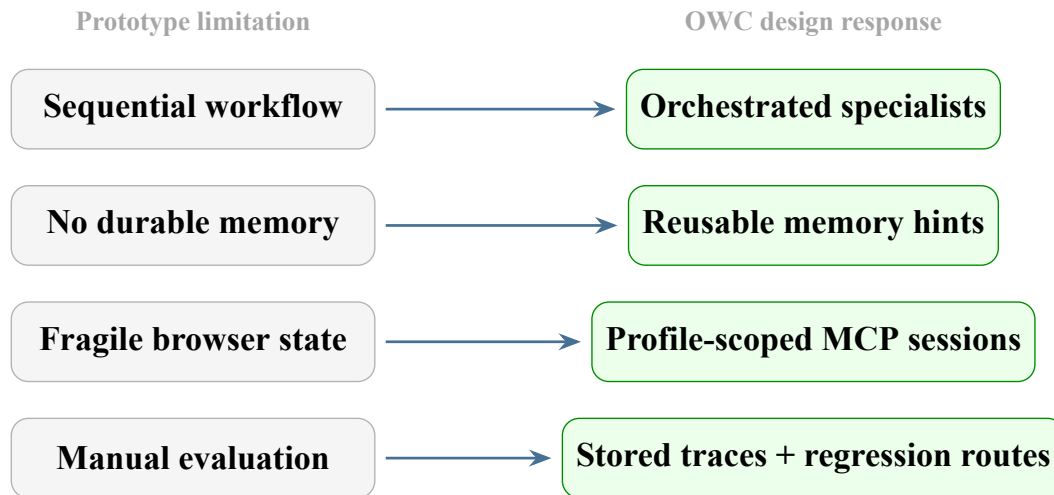


Figure 5.2: How n8n limitations shaped the Open Web Catcher implementation.

5.2 Open Web Catcher Platform

5.2.1 Runtime Boundary

OWC was designed to make the runtime inspectable: each agent, tool call, prompt compilation, screenshot, and stream candidate had to be visible after the run.

The internal name Open Web Catcher was used because the platform catches evidence from the open web without treating the task as content downloading or as a simple scraper. In this report, OWC means the complete project: FastAPI for run control and APIs, Next.js for the operator console, PostgreSQL for persistence and observability, MCP browser services for controlled page actions, and the specialist-agent runtime that coordinates classification, landing, hosting, embedded, provider, and email stages.

OWC keeps the extraction idea but makes the responsibilities explicit. FastAPI controls runs and persistence, the Next.js operator console launches and reviews investigations, MCP browser services expose controlled tools, specialist agents perform bounded tasks, and PostgreSQL stores both evidence and observability records.

5.2.2 Implementation Areas

Each implementation area was given a narrow role so that failures could be traced to the backend, UI, agent runtime, browser tools, or persistence layer instead of being treated as one generic extraction failure.

5.3 Backend and Operator Console

5.3.1 Backend Responsibilities

Area	Implementation role	Reason for the boundary
FastAPI backend	Starts jobs, exposes run detail, resolves configuration, and persists trace data [9].	Keeps runtime control and database access outside the UI.
Operator console	Launches runs, follows progress, reviews evidence, and changes settings [10].	Gives the operator one place to inspect both success and failure.
MCP browser tools	Inspect pages, click controls, capture screenshots, and harvest network evidence [11, 5].	Keeps browser actions explicit and logged.
Agent runtime	Runs classification, landing, hosting, embedded, provider, and email stages.	Splits the task into understandable responsibilities.
PostgreSQL	Stores runs, events, streams, screenshots, provider rows, emails, model calls, and tool calls.	Makes the final result auditable.

Table 5.2: Implementation areas of Open Web Catcher.

5.3.1.1 Run Control

The backend starts investigations, resolves runtime settings, registers active traces, and stores results. This keeps the operator console from owning execution logic.

The backend exposes the control surface used by the console: workflow launch, single-agent tests, active run streaming, run detail, configuration, pricing, provider lookup, and dataset execution. During implementation, the most useful backend behavior was not only starting a run; it was preserving enough runtime state to explain the run later.

5.3.1.2 Trace Assembly

Run detail is assembled from multiple records: status, events, screenshots, streams, tool calls, model usage, provider rows, and generated emails. This is why the backend became a review API, not only a job launcher.

5.3.2 Operator Console

5.3.2.1 Run Review and Evidence Navigation

The console lets the operator move from a batch summary to a single run, then from the run to screenshots, stream candidates, tool history, costs, and provider enrichment.

The operator console was built as a review tool, not as a decorative dashboard. It shows the run list, run detail, active trace, screenshots, streams, provider results, cost summaries, tool health, model settings, and dataset batches. This made evaluation faster because a failed run could be inspected from the UI without replaying the whole browser session.

5.3.2.2 Overview Telemetry

The overview tab gives a compact operational snapshot. It exposes total runs, strict stream success, stream yield, total cost, total tokens, cached tokens, LLM calls, tool success, active runs, recent runs, recent runtime events, and persistence health. These cards are not used as proof that every website works; they are used to understand the live state of the system and to connect the implementation to the evaluation vocabulary used later in the report.

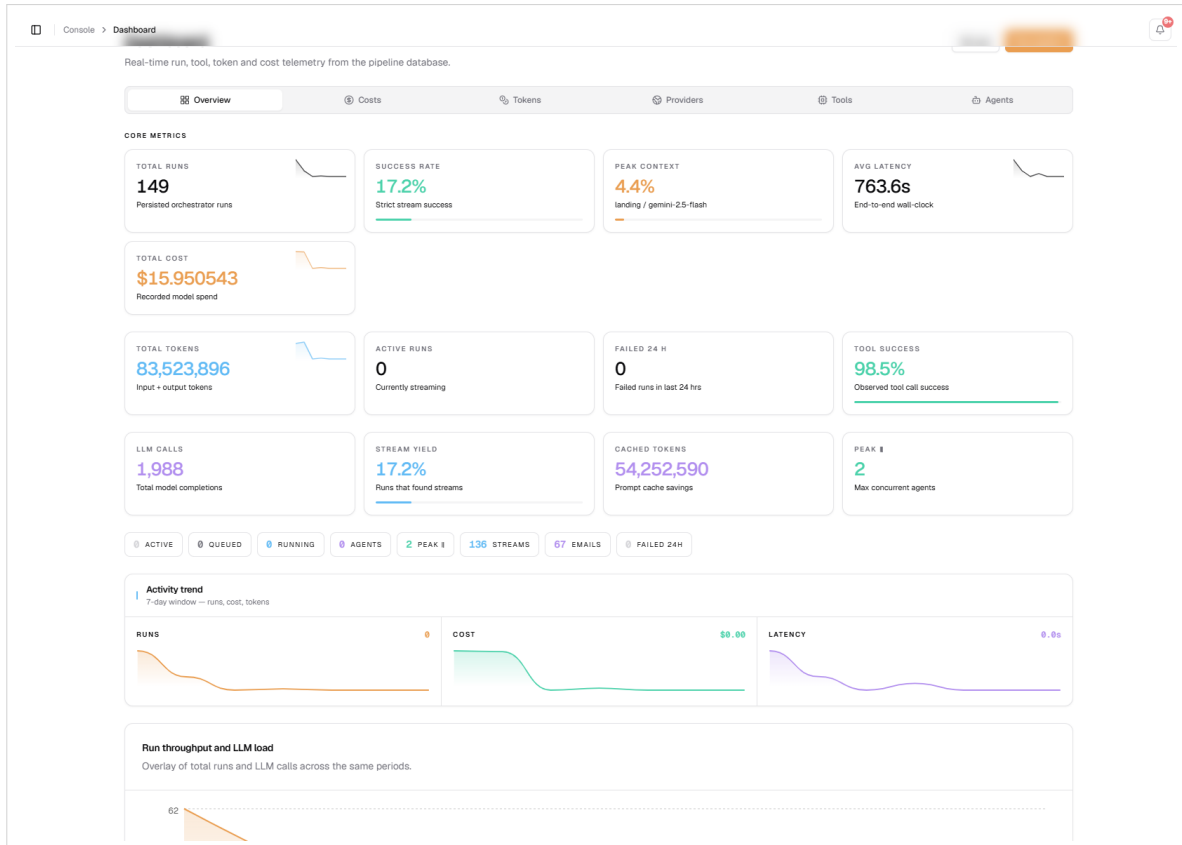


Figure 5.3: Operator console overview with run, stream-yield, token, cost, tool, and persistence telemetry.

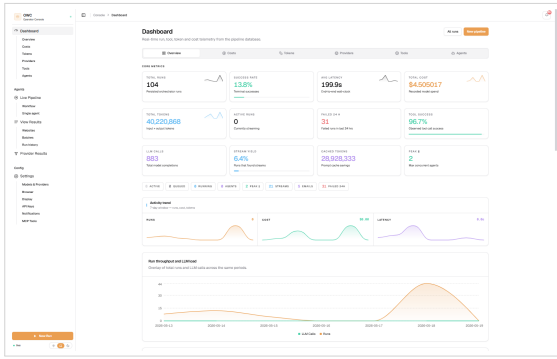
5.3.2.3 Agent Testing

The agent views are useful during development because they isolate prompt behavior, tool availability, and model settings before a full dataset batch is launched. This reduced the time needed to confirm whether a failure came from routing, a browser action, or the downstream evidence step.

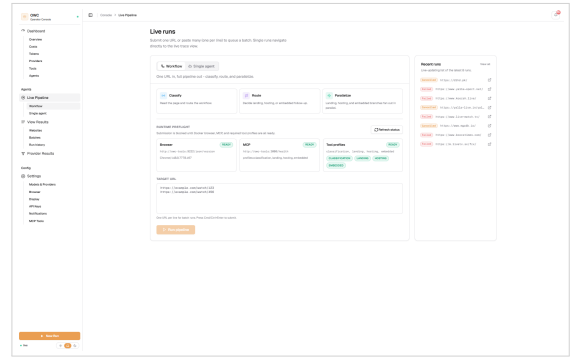
5.4 Browser Tool Layer

5.4.1 Tool Profiles

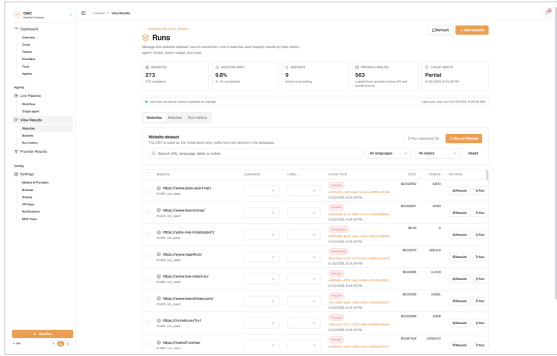
The browser tools are grouped by page role so that classification, landing, hosting, and embedded agents receive only the actions needed for their part of the investigation.



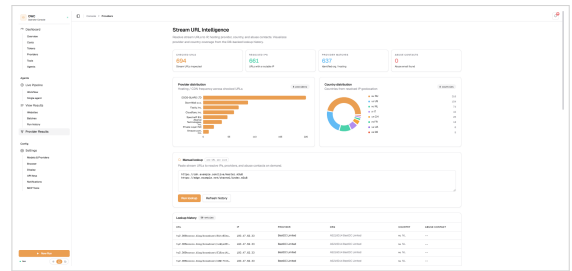
Dashboard telemetry



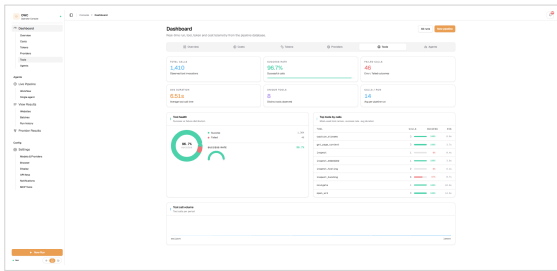
Live Pipeline launcher



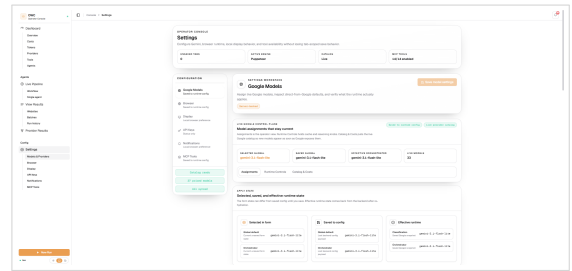
View Results dataset



Provider Results



Tool telemetry



Model settings

Figure 5.4: Current Next.js operator-console screens used for launch, review, telemetry, provider enrichment, and runtime configuration.

The browser layer uses profile-scoped tools because each agent needs a different level of control. Classification needs enough context to route the page. Landing needs candidates and visible links. Hosting needs interaction with servers and players. Embedded needs frame and media inspection. Giving every agent every tool made earlier runs less predictable.

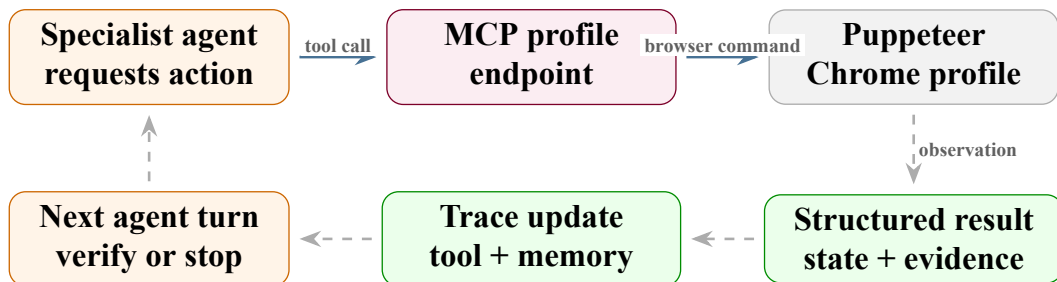


Figure 5.5: Browser tool execution loop through MCP and Puppeteer.

5.4.2 Execution Discipline

5.4.2.1 Observe Before Acting

Each browser step is expected to read the current page state before clicking or harvesting. This prevented many premature media harvests where the network was inspected before the player had actually been activated.

5.4.2.2 Verify After Acting

A successful click is not accepted only because the tool returned a success flag. The agent checks the visible page, frame tree, network entries, or screenshots before deciding that the action moved the run forward.

Tool group	Main use	Evidence returned
Inspection	Read visible text, links, buttons, frames, and page structure.	Page role hints and action candidates.
Navigation	Open target URLs, return to previous pages, and wait for page state.	Current URL, redirects, and load status.
Interaction	Click play buttons, tabs, server controls, and close popups.	Changed page or player state.
Screenshot	Capture classification, playback, or failure states.	Visual evidence linked to the run.
Harvest	Read network entries, iframe URLs, manifests, and media candidates.	Stream evidence and diagnostics.

Table 5.3: Browser tool groups used by the specialist agents.

5.5 Agents, Prompts, Memory, and Models

5.5.1 Prompt Compilation

A prompt is built from the base rules, the specialist contract, the target URL, the handoff context, runtime limits, and memory hints. This gives each agent enough context without giving it every responsibility.

Each browser-facing agent compiles a role-specific prompt from the base policy, the agent contract, runtime settings, task URL, handoff context, and memory hints. This made prompt behavior easier to revise because a landing rule did not have to be mixed with hosting-player activation rules.

5.5.2 Memory and Model Use

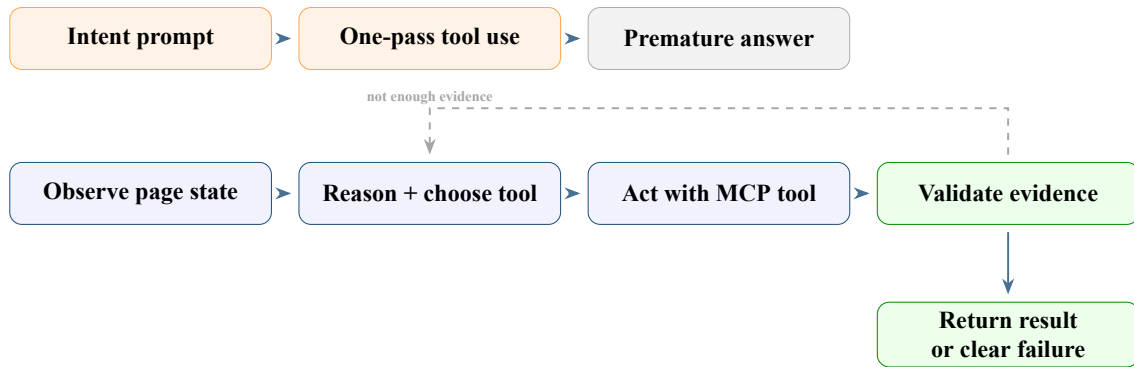


Figure 5.6: Prompt evolution from one-pass intent prompts to evidence-driven agent loops.

5.5.2.1 Memory as Hint

Memory is never treated as proof. It can help the agent try a selector or recognize a known layout, but the current browser state decides the result.

Memory is used as guidance, not as truth. A previous run can provide a useful hint about selectors, player behavior, or known layout patterns, but the current browser evidence still decides the route. This was important because streaming sites change frequently and stale memory can be worse than no memory.

5.5.2.2 Model Selection

The selected model had to support long browser traces, reliable tool calls, manageable latency, and repeated regression runs at a cost the project could observe and compare.

Model selection was treated as an operational choice. The selected model had to handle long browser traces, call tools reliably, respond quickly enough for browser loops, and keep repeated evaluation affordable. Rate limits, requests per minute, tokens per minute, and cached-input pricing therefore mattered in addition to normal answer quality [2, 3, 4].

5.6 Persistence and Observability

5.6.1 Stored Records

5.6.1.1 Evidence Records

Stream candidates, embedded URLs, screenshots, provider rows, and email drafts are stored as review outputs.

5.6.1.2 Telemetry Records

Agent runs, runtime events, model calls, tool calls, prompt compilations, and memory hints are stored to explain how the final result was produced.

The persistence layer stores the final evidence and the trace that produced it. This includes run records, agent runs, runtime events, screenshots, stream records, provider analyses, email

drafts, model usage, tool calls, prompt compilations, and memory hints. These rows made it possible to review failures instead of treating a failed run as an opaque error.

Stored record	Example question answered	Why it matters
Runtime event	Which stage failed and why?	Supports failure diagnosis.
Screenshot	What did the browser actually show?	Supports visual review.
Tool call	Which browser action was attempted?	Separates prompt errors from tool/runtime errors.
Model call	Which model was used and how many tokens were spent?	Supports cost and model evaluation.
Provider row	Which host or network was behind a stream URL?	Supports rights-enforcement review.

Table 5.4: Observability records used during run review.

5.7 Deployment Direction

5.7.1 Company Deployment Flow

5.7.1.1 agent_tasks Input Queue

In the company deployment decision, target URLs are sent to an Azure Service Bus queue named `agent_tasks`. These URLs are usually landing pages and include the metadata needed to start an investigation. The queue boundary keeps the caller separate from the browser execution time, which is important because one investigation can take several minutes.

5.7.1.2 n8n and Puppeteer Container

The deployment unit runs the n8n workflow together with the Puppeteer tool service and browser instances. Keeping the workflow and browser tooling in the same container avoids a slow request-response path and makes long browser investigations easier to schedule. The container consumes one task, opens the browser context, executes the workflow, and collects matched URLs and metadata.

5.7.1.3 agent_results Output Queue

When a run finds matches, the container sends the matched URLs and their metadata to a second Service Bus queue named `agent_results`. This keeps the evidence handoff asynchronous and lets downstream company systems consume results separately.

Azure Service Bus is a good fit for this deployment style because browser investigations can be slow, variable, and easier to retry when they are handled as queued work instead of synchronous web requests [13].

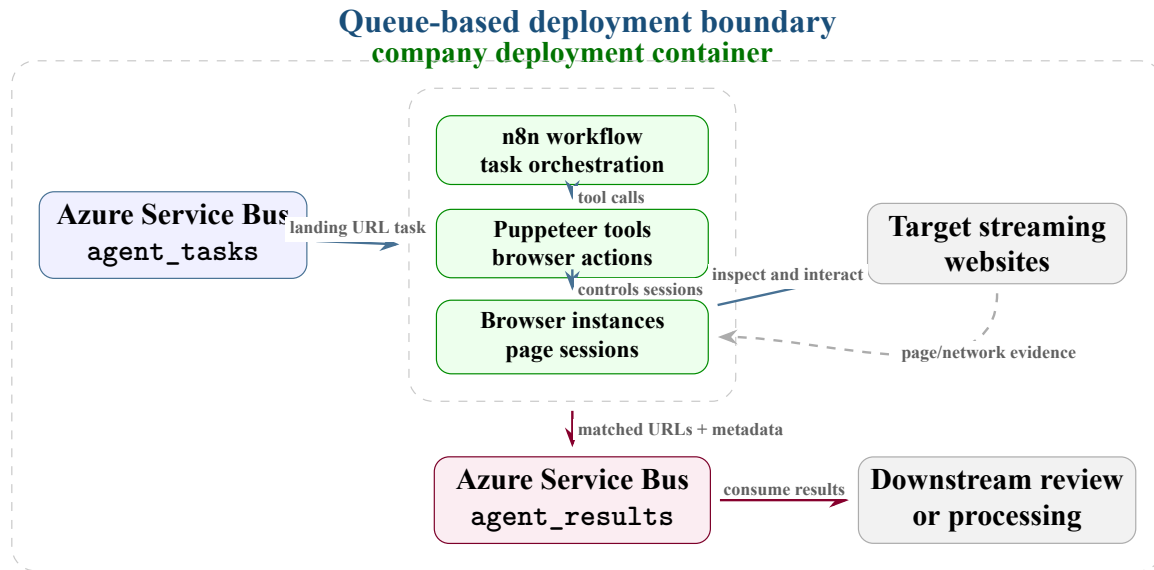


Figure 5.7: Company deployment flow using agent_tasks for input URLs and agent_results for matched URLs with metadata.

5.7.2 Relation to Open Web Catcher

The company deployment path uses the validated n8n container flow for queued URL processing. OWC documents and implements the maintainability direction: clearer agents, richer observability, typed persistence, and repeatable evaluation.

The operational deployment scope therefore remained focused. The n8n workflow continued to serve the company deployment decision during the internship, while OWC defined and implemented the maintainability path that could be adopted progressively.

5.8 Implementation Summary

The implementation moved from a working prototype toward the reviewable OWC platform. The main engineering gain was not a single extraction script. It was a runtime where prompt changes, browser tools, screenshots, costs, stream evidence, provider data, and failures could be inspected together. That foundation is what made the evaluation in the next chapter possible.

Evaluation and Testing

The system was evaluated as an evidence-collection workflow, not as a simple classifier. A run is useful only when it reaches a reviewable state: the page is classified, the agent follows the right page path, playback or embedded evidence is inspected, at least one stream or media candidate is collected when available, and screenshots or traces explain the outcome.

This chapter therefore answers six questions. What was tested? Why did those tests matter? How was success defined? What improved across versions? What still failed? How did the evaluation support the internship contribution?

6.1 Evaluation Context

6.1.1 Dataset and Snapshot

The evaluation used website-level runs because the system can classify a page correctly and still fail later during hosting or embedded-player evidence collection.

The main test object was a regression batch of more than 200 streaming websites. The batch mixed landing pages, hosting pages, embedded players, redirects, popup-heavy pages, tokenized streams, and several language/layout families. The same batch was rerun after prompt, tool, and model changes so that improvements could be compared against previously working sites.

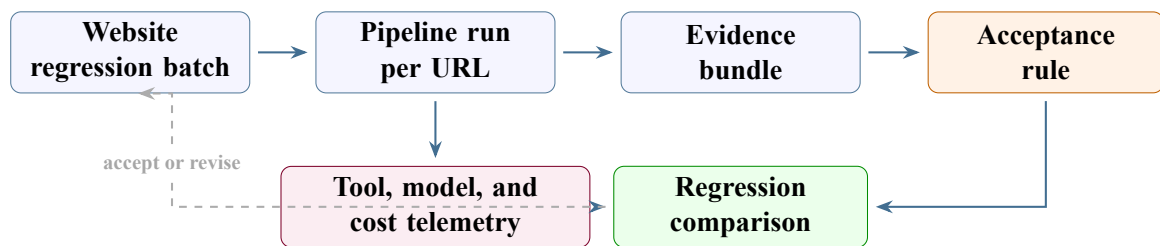


Figure 6.1: Evaluation protocol used for website-batch regression testing.

The regression approach was necessary because a browser-agent change can fix one website while breaking another. A new prompt that clicks more aggressively can activate a difficult player, but it can also lose page focus after an advertising popup. A tool update can improve media harvesting, but if it runs before playback is attempted, it returns weak evidence. For that reason, each change was judged against old working cases and new difficult cases together.

6.1.2 Dashboard Snapshot

The operator console also exposes a live dashboard snapshot. These values are not the same denominator as the controlled regression batch. They describe the state of persisted runs and

Evaluation item	Value used in the report
Evaluation snapshot	May 2026 report snapshot.
Dataset size	More than 200 streaming websites, selected to cover different page families and runtime obstacles.
Execution unit	One pipeline run per target URL, with classification, specialist routing, browser-tool execution, evidence capture, and persistence.
Main runtime model	google_genai::gemini-3.1-flash-lite.
Prompt/evaluation version	Final regression prompt version after five prompt/tool iterations.
Acceptance rule	A website is counted as working only when the run produces reviewable evidence, not merely a completed status.
Metric sources	Batch results, operator-console telemetry, persisted tool/model rows, provider-enrichment rows, screenshots, and manual spot checks.
Review type	Batch-derived for success rates, dashboard-derived for cost/tool telemetry, and manual for representative evidence validation.

Table 6.1: Evaluation context for the website regression batch.

telemetry at the time the report was checked, while the regression tables describe selected evaluation batches and manual acceptance rules.

Metric family	Snapshot values	How it should be read
Run history	149 total runs, 25 strict successes, 8 partials, 6 no_streams, 14 no_hosting_pages.	Persisted operational history, not the controlled regression success table.
Stream yield	136 streams, 67 emails, 17.24% strict stream success.	Live dashboard evidence yield across persisted terminal runs.
Model usage	83,523,896 total tokens, 1,988 LLM calls, \$15.950543 total recorded spend.	Dashboard aggregate of model usage rows.
Cache visibility	54,252,590 cached input tokens shown by the dashboard headline.	Useful for cost analysis, but not forced into the same denominator as every model breakdown row.
Tool telemetry	3,113 observed tool calls, 3,066 successful calls, 47 failed calls, 98.49% observed tool success.	Runtime health indicator for browser and memory tools.
Persistence health	697 agent runs, 18,094 runtime events, 1,947 persisted LLM calls, and 3,113 tool calls.	Evidence that the review surface is backed by normalized records.

Table 6.2: Dashboard snapshot used as operational telemetry, separate from regression-batch metrics.

6.2 Success Metrics

6.2.1 Acceptance Rule

A run is successful only when it gives a reviewer enough evidence and trace context to understand what happened.

The acceptance rule was intentionally stricter than a normal “run completed” status. A completed run can still be unusable if it stopped on a listing page, missed host candidates, took a screenshot before player activation, or returned an unrelated network URL.

Metric	Definition used during evaluation
Website success rate	Percentage of evaluated websites that satisfy the full acceptance rule.
Classification accuracy	Percentage of URLs routed to the correct page type: landing, hosting, embedded, blocked, or other.
Host-candidate recall	Share of expected host candidates discovered on landing pages.
Player activation success	Share of hosting/embedded attempts where the player state changed or a meaningful playback blocker was recorded.
Stream evidence extraction rate	Share of runs returning a useful manifest, media, iframe, network, or embedded URL candidate.
Provider coverage	Share of checked stream or host URLs enriched with provider or network information.
Tool success rate	Successful browser/tool calls divided by total browser/tool calls.
Regression retention	Share of previously working websites that remained working after a prompt, model, or tool change.
Cost per working website	Estimated model spend divided by websites satisfying the acceptance rule.
Cache hit ratio	Cached input tokens divided by total input tokens considered by the cache view.

Table 6.3: Evaluation metrics used for agent, model, tool, and cost analysis.

Two formulas guided most regression decisions:

$$WebsiteSuccessRate = \frac{working_websites}{evaluated_websites}$$

$$RegressionRetention = \frac{previously_working_websites_still_working}{previously_working_websites}$$

The objective was not to maximize tool calls or produce longer reasoning. The objective was to improve the number of websites that reached evidence while keeping previously working websites stable.

6.3 Agent Regression Results

6.3.1 Prompt and Tool Iterations

The largest improvement came from forcing the browser agent to inspect the page, perform one relevant action, and verify the changed state before harvesting evidence.

The strongest improvement came from changing the agents from broad intent-based instructions to a controlled observe–act–verify loop. Early runs often stopped after reading the first page, harvested before activating playback, or accepted a weak screenshot as final evidence. Later runs forced the agent to inspect the page state, perform one browser action, verify the result, and only then decide whether to continue or stop.

Version	Main change tested	Regression focus	Working sites
v1	Intent-based prompt.	Early stop, weak tab navigation, and premature harvest.	38%
v2	Observe–act–verify loop with explicit tab checks.	Navigation recall and screenshot-grounded decisions.	68%
v3	Popup handling and JavaScript click fallback.	Server activation after ad traps.	74%
v4	Mandatory media-state check before harvest.	Player activation and stream evidence timing.	81%
v5	Tool-budget cap and early success exit.	Cost reduction without losing previously working websites.	83%

Table 6.4: Prompt regression history measured on the website batch.

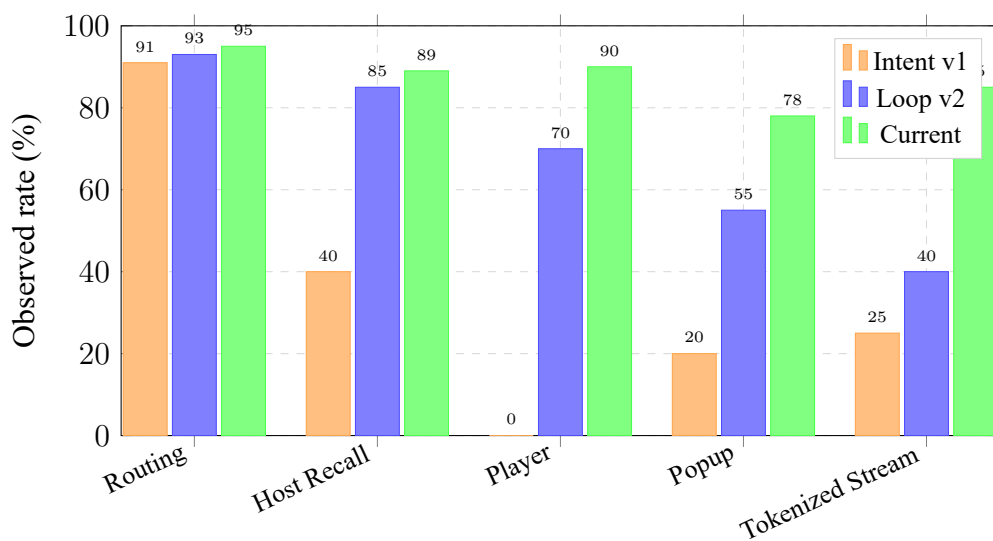


Figure 6.2: Regression-batch improvement across the main browser-agent KPIs.

Metric	Intent v1	Loop v2	Current
Page-type routing accuracy	91%	93%	95%
Host-candidate recall	40%	85%	89%
Player activation success	0%	70%	90%
Popup-interference recovery	20%	55%	78%
Tokenized-stream recovery	25%	40%	85%
Manifest recovery on AlbaPlayer case	0%	70%	90%
Manifest recovery on tokenized Clappr case	—	40%	85%

Table 6.5: Main browser-agent KPIs after prompt and tool iterations.

The results show why evaluation had to be website-level. A classification model can appear accurate while the full investigation still fails because the landing agent misses host candidates or because the hosting agent harvests too early. The final prompt/tool version improved the full path: routing, candidate discovery, player activation, popup recovery, and tokenized-stream handling.

6.4 Model, Cost, and Cache Behavior

6.4.1 Runtime Model Choice

The model choice was judged by browser-agent behavior, tool-use stability, context length, latency, and repeated-batch cost.

The final runtime used `google_genai::gemini-3.1-flash-lite`. The choice was based on operating behavior rather than one benchmark score. The workflow needed long context for browser traces, predictable tool use, low latency during interactive browser actions, and a cost profile that could survive repeated website batches. Pricing, context caching, and rate-limit concepts were checked against the provider references used by the implementation [3, 4, 2, 1].

The detailed token counters are kept separate because the operator console groups usage by different dashboard views. The cost view, cache view, and headline token card may not share exactly the same denominator when a run has partial usage rows or provider aliases. For the report, the important interpretation is therefore not a forced grand total; it is the split between input, cached input, output, calls, and spend. That split showed that prompt and tool-context caching mattered for large repeated batches.

Metric	Interpretation	Observed value
Selected model	Default model for the final regression snapshot.	Gemini 3.1 Flash-Lite
Total model spend	Sum of recorded or estimated model usage rows in the dashboard snapshot.	\$15.950543
Average cost per run	Dashboard spend normalized by terminal runs with recorded usage.	\$0.110004
LLM calls	Model invocations shown by the dashboard aggregate.	1,988
Input tokens	Prompt, trace, tool-context, and memory tokens recorded by the dashboard aggregate.	82,506,629
Cached input tokens	Cached input shown by the dashboard headline card.	54,252,590
Output tokens	Generated model tokens in the dashboard aggregate.	1,017,267
Total tokens	Input and output tokens shown by the overview tab.	83,523,896

Table 6.6: Model, token, cost, and cache telemetry used during evaluation.

6.5 Provider and Evidence Enrichment

6.5.1 Infrastructure Context

Provider enrichment adds infrastructure context to stream evidence, but it does not replace human validation.

Provider analysis runs after stream or embedded URLs are collected. It resolves hostnames, checks whether routable IP addresses are available, and enriches the result with provider or registration data through IP and RDAP/WHOIS sources [14, 15]. This step does not prove infringement by itself, but it makes the extracted evidence more useful for review because a media URL can be linked to infrastructure context.

The observed provider distribution included infrastructure such as TECHOFF SRV LIMITED, Onehostplanet, IP Volume, BestDC, Private Layer, Altrosky, Telkom, Amazon, and Tempest Hosting. Those values are used as review context, not as final legal conclusions.

6.6 Tool Evaluation

6.6.1 Tool Health and Usefulness

A tool call can technically succeed and still be badly timed. Evaluation therefore checked whether the tool helped the evidence path, not only whether it returned without an error.

Tool evaluation checked both technical success and behavioral usefulness. A screenshot call can succeed while still being useless if it captures a page before player activation. A network

Provider metric	Meaning	Observed value
Provider analysis records	Stream or media evidence rows enriched by the provider stage.	136
Runs with stream evidence	Runs that produced at least one stream candidate in the dashboard snapshot.	25
Total stream records	Persisted stream records shown by the overview tab.	136
Generated email drafts	Review drafts generated from stream/provider evidence.	67
Unique provider names	Distinct provider names shown in the overview breakdown.	12
Largest provider group	Most frequent provider in the snapshot.	TECHOFF SRV LIMITED, 38 analyses

Table 6.7: Provider-enrichment telemetry for extracted stream evidence.

harvest can succeed while returning no evidence if it runs before a stream is created. The tool layer was therefore evaluated with the same website acceptance rule used for agents.

Tool finding	Evaluation meaning	Observed value / fix
Observed tool calls	Browser/tool invocations with persisted outcome rows.	3,113
Successful calls	Calls that returned a successful tool outcome.	3,066
Failed calls	Calls that returned an error or failed outcome.	47
Tool success rate	Successful observed calls divided by observed tool calls.	98.49%
Average duration	Mean observed duration per tool call.	7.636 s
Calls per run	Approximate observed browser/tool actions per persisted run.	20.9
Premature harvest	harvest() ran before playback produced media evidence.	Activation guard and media-state check.
Wrong tab/focus	Agent reasoned over an advertising tab instead of the player tab.	Popup close and focus restore.
Navigation misuse	Same-page controls were opened with navigation instead of click interaction.	Clearer interact versus navigate policy.

Table 6.8: Tool-health telemetry and main tool-regression fixes.

The dashboard success rate confirmed that the tool layer was mostly stable, but the regression findings show that stability is not enough. The important engineering work was aligning tool

timing with page behavior: inspect before acting, interact before harvesting, restore focus after popups, and take screenshots at evidence moments rather than as repeated waiting.

6.7 Remaining Limitations

6.7.1 Known Failure Sources

The evaluation is a snapshot of a changing target space. A result that worked during the batch can fail later if the website, player, token policy, or protection layer changes.

Several limitations remain. Some sites change their JavaScript or anti-bot behavior between runs. Tokenized streams can expire before enrichment finishes. Deep iframe chains still require careful budget management. Provider lookups do not always expose abuse contacts. Finally, the reported numbers are a regression snapshot, not a guarantee that every future streaming site will behave the same way.

These limitations are why the final system should be presented as a reference architecture and evaluation foundation. It improved evidence collection and made failures inspectable, but it still needs larger labeled batches, more repeated runs, and production-scale monitoring before it can be treated as a complete enforcement platform.

6.8 Chapter Summary

The evaluation supports the internship contribution because it measures the complete investigation path. The system improved from weak intent-based behavior to a controlled browser-agent workflow with higher website success, clearer tool discipline, measurable model cost, cache visibility, and provider enrichment. Most importantly, the evaluation method gives future work a practical rule: a change is accepted only when it adds coverage without losing evidence that already worked.

Conclusion and Perspectives

7.1 General Conclusion

The internship started from a practical evidence problem. Illegal sports streaming websites rarely expose useful proof through static HTML. They rely on dynamic listings, JavaScript tabs, ad popups, nested iframes, short-lived stream URLs, redirects, and provider indirection. A normal scraper can collect links, but it cannot reliably explain how a stream was reached or why a player failed to activate.

My mission was to study that problem and build a technical foundation for agentic evidence collection. The work began with an n8n prototype because it was the fastest way to validate the idea with real pages. That prototype proved that an LLM-guided browser workflow could inspect pages, click through players, and recover evidence, but it also showed the limits of long visual workflows: difficult debugging, weak regression control, limited observability, and poor separation between page types.

The final contribution was Open Web Catcher (OWC), the complete evidence-collection platform that turned those lessons into a structured workflow. OWC separates page classification, landing exploration, hosting interaction, embedded-player inspection, provider enrichment, and email drafting. It also preserves screenshots, runtime events, tool calls, model usage, cost fields, provider rows, and final evidence so that a run can be reviewed after execution.

7.2 Objectives Achieved

The main objectives defined at the beginning of the report were achieved at the reference-architecture level. The system can ingest URLs, classify page types, route to specialist agents, operate browser tools, capture evidence artifacts, enrich stream hosts, and expose results through an operator console. The evaluation chapter also showed that progress was measured through regression batches and dashboard telemetry instead of isolated demonstrations.

The achieved result is an engineering foundation with validated behavior and measurable traces. It should not be read as a claim that every hostile website is permanently handled. The value is that the workflow gives the company a clearer way to build, inspect, and improve evidence collection.

7.3 Main Contributions

The first contribution was the feasibility study. It reduced uncertainty around the central question: can an agent use a browser to collect evidence from streaming sites that require inter-

Objective	Result achieved during the internship
Study the evidence problem	Identified why landing, hosting, and embedded pages need different browser strategies.
Build a feasibility prototype	Implemented and tested an n8n workflow that validated browser-agent evidence collection on real dynamic pages.
Design a maintainable architecture	Created OWC with FastAPI, Next.js, a specialist-agent runtime, MCP browser tools, PostgreSQL persistence, and an operator console.
Make runs reviewable	Stored traces, screenshots, streams, provider rows, model usage, tool calls, and failure reasons.
Evaluate changes safely	Used website regression batches, KPI tracking, model cost telemetry, provider coverage, and tool-health metrics.

Table 7.1: Summary of achieved objectives.

action? The answer was positive, but only when the agent was constrained by page type, tool order, and evidence checks.

The second contribution was the specialist-agent design. Instead of giving one large agent every responsibility, the final system assigns separate roles to classification, landing, hosting, embedded, provider, and email stages. This made prompts shorter, failures easier to diagnose, and evaluation more precise.

The third contribution was observability. Each run preserves enough information to understand the path taken by the system: which page type was detected, which agent acted, which tools were called, which screenshots were captured, which streams were found, and how much the model calls cost. This is essential for an evidence workflow because a final URL without context is not enough.

The fourth contribution was the regression-based evaluation method. The project does not claim that a single successful run proves reliability. It evaluates changes by checking whether more websites work without breaking those that already worked.

7.4 Technical and Personal Learning

The internship strengthened several engineering skills. Browser automation was the first major learning area: dynamic pages require real interaction, focus management, iframe inspection, network tracing, and screenshot validation. Agent design was the second: prompts must guide actions without letting the model drift across responsibilities. Prompt engineering also became practical rather than theoretical, because each prompt change was judged by whether it fixed or broke real websites.

I also learned the importance of observability. During development, many failures looked similar at first: no stream found, player not active, empty network harvest, or incorrect page route. Persisted events, screenshots, tool results, and model usage made those failures under-

standable. Finally, the cost and cache work showed that an AI workflow is not only about accuracy; it must also remain affordable and repeatable across batches.

7.5 Limitations

7.5.1 Runtime Instability

The result should be understood as a strong foundation, not a complete production enforcement platform. Streaming sites change frequently, and anti-bot behavior can invalidate a path that worked earlier. Some pages still require human review before evidence can be trusted. Tokenized URLs can expire quickly. Deep iframe chains and popup-heavy pages still require careful budgets, retry rules, and manual validation.

7.5.2 Evaluation Scope

The evaluation numbers are also a snapshot. They are useful because they were produced through repeated website batches and persisted dashboard telemetry, but they should be expanded with more labeled websites, longer monitoring periods, and clearer manual ground truth before being treated as final production accuracy.

7.6 Future Work

7.6.1 Evaluation and Regression

The first priority is to expand the benchmark set with labeled examples for landing pages, hosting pages, embedded players, popups, redirects, tokenized streams, and blocked pages. The second priority is to automate regression suites for each specialist agent so that prompt, model, and tool changes can be tested before they affect a larger batch.

7.6.2 Candidate Ranking and Memory

The system should improve candidate ranking. When a landing page exposes many host links, the agent should spend less time on dead or low-quality candidates. Memory can help here by reusing selectors, server labels, and provider patterns discovered during previous successful runs without hardcoding one website.

7.6.3 Deployment and Review

On the deployment side, long browser executions should move toward queue-driven workers while the operator console remains responsive. A useful review feature would be screenshot annotation. Since the system already captures screenshots, reviewers could mark player controls, server buttons, blocking overlays, broadcaster logos, iframe regions, and evidence areas directly on the captured image. These annotations would make human validation faster and would improve evidence handoff.

7.7 Final Remarks

The project demonstrates that agentic browser automation can support sports streaming evidence collection when it is constrained, observable, and evaluated through regression. The final value is not that the system permanently handles every website. The value is that it turns an unstable manual investigation into a structured workflow: classify the page, route to the right specialist, interact deliberately, verify player state, extract evidence, preserve the trace, enrich providers, estimate cost, and learn from failures.

This gives Soft Stars a concrete foundation for continuing the work with larger datasets, stronger monitoring, more robust deployment, and clearer operational review for the beIN SPORTS use case.

Evaluation Case Studies

The evaluation chapter summarizes the batch results. This appendix gives more detail on representative cases that shaped the final design. The cases are not included to claim that every streaming site behaves the same way. They show the kind of failures that appeared during testing and the engineering response used to make the workflow more stable.

A.1 Case Study A: Landing Page With Repeated Event Cards

Some websites expose the first useful evidence on a discovery page rather than on a player page. The page lists matches, leagues, or channels, and each event may link to one or more downstream hosting pages. The first prototype often stopped after collecting a few visible links because it treated the landing page as a normal link list.

Evaluation point	Observation
Initial symptom	The agent returned only the first visible match cards and missed links hidden under date tabs or lower page sections.
Main risk	Host-candidate recall looked acceptable on small pages but failed when the same layout repeated by date, league, or event block.
Change introduced	The landing agent was required to inspect visible sections, open date or category tabs, scroll when needed, and keep a ledger of already checked blocks.
Acceptance signal	The run produced a ranked list of hosting candidates with the associated match context and no duplicate navigation noise.

Table A.1: Landing-page regression case: repeated event cards.

This case changed the way landing pages were evaluated. A run could not be counted as working simply because it found one event. It had to show that the visible discovery surface was inspected deliberately. That is why host-candidate recall became a separate KPI in Chapter 6.

A.2 Case Study B: Hosting Page With Popup Interference

Hosting pages were usually harder than landing pages because evidence appears only after interaction. The agent has to click play, switch servers, close popups, restore focus, and check whether the player state changed. In early runs, the agent often clicked a play button, landed on an advertisement tab, and continued reasoning from the wrong page.

Evaluation point	Observation
Initial symptom	The browser opened a popup or new tab after the first click, and the agent interpreted the advertisement page as the current investigation target.
Main risk	The final trace looked active because tools were still executing, but the evidence path had left the original player page.
Change introduced	After every interaction, the browser layer checks for new contexts, closes unrelated popups, restores focus to the original page, and retries the click with a safer strategy.
Acceptance signal	The screenshot and media-state check both refer to the original hosting page or to a controlled embedded fallback, not to an advertising tab.

Table A.2: Hosting-page regression case: popup and tab focus recovery.

This case explains why screenshot evidence is not optional. A network result alone does not prove that the agent stayed on the correct page. The screenshot shows whether the player page, popup, overlay, or blocker was visible at the moment the claim was made.

A.3 Case Study C: Embedded Player With Delayed Manifest

Embedded pages sometimes look simple because they contain only one player. However, the stream manifest may not be requested until after a play action, after a short delay, or after an iframe redirect. Early harvesting often returned no media evidence because the tool ran before the player produced the network requests.

Evaluation point	Observation
Initial symptom	The embedded agent called the harvest tool immediately and returned an empty stream list.
Main risk	The site was incorrectly marked as unsupported even though the manifest appeared after player activation.
Change introduced	The embedded agent now inspects player readiness, clicks play when needed, waits for network activity, captures a screenshot, and only then harvests media evidence.
Acceptance signal	The run returns a manifest or media request after an observed player-state change, or it records a clear failure reason if activation is blocked.

Table A.3: Embedded-player regression case: delayed manifest generation.

The lesson from this case was that tool order matters as much as tool availability. The same harvest tool can be ineffective or useful depending on when it is called. This is why the final prompt requires observe–act–verify steps before extraction.

A.4 Case Study D: Tokenized Stream URL

Some players generate stream URLs that include short-lived tokens. The URL can work during the run and become invalid later. This behavior created two requirements: preserve the URL exactly as observed, and preserve enough context to understand where it came from.

Evaluation point	Observation
Initial symptom	The run found a media URL, but a later manual replay failed because the token had expired.
Main risk	A reviewer could mistake token expiry for a false extraction result.
Change introduced	The evidence bundle stores the URL, timestamp, page URL, frame source, screenshot, provider lookup, and failure context.
Acceptance signal	A tokenized stream is accepted only with trace context: where it was observed, when it was captured, and which page state produced it.

Table A.4: Stream-evidence case: tokenized URL preservation.

This case also influenced provider enrichment. A tokenized URL should be enriched quickly while its host and path are still visible in the run context. Even if the token later expires, the provider and network evidence remain useful for review.

A.5 Case Study E: Wrong Page Classification

Classification failures were usually not caused by the model misunderstanding a single word. They were caused by pages that mixed several roles: a landing page with an embedded player, a hosting page with a match list, or an error page with old player markup still present in the DOM.

Evaluation point	Observation
Initial symptom	The classifier routed a mixed page to the wrong specialist, which caused the next agent to use unsuitable tools.
Main risk	A landing agent may collect too many irrelevant links from a player page, while a hosting agent may click controls that do not exist on a listing page.
Change introduced	Classification now considers visible page role, player signals, iframe context, and action needed, rather than URL shape alone.
Acceptance signal	The chosen route gives the next specialist a task that can produce useful evidence or a defensible failure reason.

Table A.5: Routing case: mixed-role page classification.

This case justified the handoff object in the architecture chapter. The orchestrator does not only pass a URL. It passes the page role, route source, evidence checklist, candidate context, and navigation policy.

A.6 Case Study F: Provider Enrichment Without Abuse Contact

Provider enrichment was useful even when it did not return an abuse contact. Some hosts resolve to a provider or network owner, but the lookup path does not expose a direct abuse email. Earlier reports treated that as a failure. The final evaluation separates provider coverage from abuse-contact discovery.

Evaluation point	Observation
Initial symptom	A stream URL resolved to an IP address and provider, but no abuse email was returned.
Main risk	The evidence enrichment looked like a total failure even though the provider and network context were useful.
Change introduced	The dashboard and evaluation split checked URLs, resolved IPs, provider matches, and abuse contacts into separate metrics.
Acceptance signal	Provider coverage is counted when provider or network context is found; abuse-contact discovery remains a separate downstream field.

Table A.6: Provider-enrichment case: useful metadata without abuse contact.

This distinction makes the evaluation more honest. The project can show that provider enrichment works for most extracted URLs while still admitting that abuse-contact discovery needs additional sources and manual review.

A.7 Case Study Summary

Table A.7: Summary of evaluation case studies and design impact.

Case family	Failure pattern	Design response
Landing discovery	Visible cards were collected, but hidden date tabs or lower sections were missed.	Landing-specific inspect, checked-section ledger, and host-candidate recall metric.
Hosting interaction	Popup or ad tab stole focus after a click.	Popup closing, focus restoration, screenshot checkpoint, and safer click strategies.
Embedded extraction	Harvesting ran before the player emitted media requests.	Activation guard, media-state check, and delayed harvest.

Case family	Failure pattern	Design response
Tokenized streams	URL became invalid after the run.	Timestamped stream record, screenshot context, provider enrichment, and trace preservation.
Mixed-role pages	Classifier chose the wrong specialist.	Route based on page role and required action, not only URL pattern.
Provider enrichment	Provider found, but abuse contact missing.	Separate metrics for checked URLs, resolved IPs, provider matches, and abuse contacts.

These cases explain why the final evaluation is regression-oriented. Each fix improved one behavior, but every fix also had to be checked against pages that already worked. The case studies therefore connect implementation decisions to the evaluation metrics used in the main report.

A.8 Manual Review Checklist for Case Studies

For each representative case, the manual review followed the same questions. This made the case studies comparable and avoided accepting a run because it looked successful at first glance.

- Was the initial page role visible in the screenshot or DOM signals?
- Did the selected specialist match the page role and evidence objective?
- Did the trace show the browser action that changed the page state?
- Did screenshots support the final claim or the failure reason?
- Was stream evidence captured after player activation rather than before it?
- If provider enrichment failed, was the missing field explicit?
- Did the fix improve this case without breaking older successful cases?

The checklist is intentionally simple. Its purpose is to keep evaluation grounded in evidence that a reviewer can inspect: page role, action, state change, stream result, provider context, and regression impact.

Agent Prompt Contracts

This appendix records the prompt contracts used by the agent runtime. They are included because prompt behavior was part of the engineering work: each agent needed a bounded role, a clear tool policy, a stopping rule, and a structured result. The versions shown here correspond to the final observe–act–verify behavior after the regression iterations described in Chapter 6.

B.1 Orchestrator Agent Prompt

Listing B.1: Orchestrator agent prompt contract (final version).

```
ROLE:
You are the run orchestrator for an evidence-collection pipeline.
You do not browse pages directly. Your role is to classify the seed URL,
prepare handoffs, route to the correct specialist agents, and aggregate
the final PipelineResult.

OBJECTIVE:
Convert one target URL into a reviewable run result:
- page classification,
- specialist extraction outputs,
- stream or embedded evidence when available,
- screenshots,
- provider analysis,
- email draft when evidence exists,
- failure reason when evidence cannot be collected.

ROUTING RULES:
1. Always start with ClassificationAgent.
2. If the page is landing_page:
    route to LandingPageAgent and collect hosting candidates.
    For ranked candidates, route each useful candidate to HostingPageAgent.
3. If the page is host_page:
    route directly to HostingPageAgent.
4. If the page is embed_video_page:
    route directly to EmbeddedPageAgent.
5. If a hosting run returns embed_url but no streams:
    route that embed_url to EmbeddedPageAgent.
6. If streams exist:
    run provider enrichment and create reviewable email draft.
7. If no streams exist:
    skip provider/email stages and return a failure result with trace.

HANDOFF CONTENT:
```

Each specialist receives:

- target_url,
- page_type,
- route_source,
- evidence_needed,
- known_candidates,
- memory_hints,
- navigation_policy,
- current_tool_budget.

STOPPING RULES:

- Stop when a PipelineResult contains sufficient evidence.
- Stop when all candidate routes are exhausted.
- Stop when budget limits are reached.
- Never hide failure: return the responsible stage and reason.

B.2 Classification Agent Prompt

Listing B.2: Classification agent system prompt (final version).

ROLE:

You are a page classification specialist. Your only task is to determine the type of web page currently loaded in the browser.

OBJECTIVE:

Return a single classification result: landing_page | host_page | embed_video_page | other.

TOOLS AVAILABLE:

- inspect_page: retrieve page title, meta description, visible links, iframe list, and player signals from the current DOM state.
- screenshot: capture a visual snapshot of the current page.
- get_frame_tree: enumerate top-level and nested frame URLs.

PROCEDURE:

1. Call inspect_page to gather structural signals.
2. Call screenshot to visually confirm the page layout.
3. Reason about the evidence:
 - landing_page: shows match schedule, team cards, event listings with links to match pages.
 - host_page: single match focus, server switcher buttons, visible player container, embedded iframe with albaplayer/clappr/hls.js.
 - embed_video_page: minimal UI, one player fills the viewport, no navigation, origin is an iframe src.
 - other: ads, login walls, error pages, irrelevant content.
4. Return JSON:

```
{ "page_type": "...", "confidence": 0.0-1.0, "reasoning": "..." }
```

IMPORTANT:

- Do NOT call more than 3 tools total.
- Do NOT perform any interactions.
- Base your decision on DOM evidence and the screenshot, not URL patterns alone.

B.3 Landing-Page Agent Prompt

Listing B.3: Landing-page agent system prompt (final version).

ROLE:

You are a landing-page extraction specialist. Your task is to discover all hosting-page URLs and match metadata from a sports streaming listing page.

OBJECTIVE:

Return a ranked list of hosting-page URLs with associated match metadata (teams, time, league, broadcaster if visible).

TOOLS AVAILABLE:

- inspect_landing: extract visible match links, section labels, date tab structure, and broadcaster references.
- interact: click date tabs, scroll, reveal hidden sections.
- screenshot: capture state after each interaction.

REACT LOOP PROCEDURE:

OBSERVE: call inspect_landing on the current page state.

THINK: identify which links are hosting candidates (URL contains "matches", "watch", "live", "stream" or links to specific match pages).

Identify whether any date tabs or "load more" controls exist.

ACT: if unchecked tabs/sections exist, click them and re-observe.

REFLECT: have all visible sections been checked? If yes, compile results.

REPEAT until all sections checked or tool budget exhausted.

OUTPUT FORMAT:

```
{
  "hosting_candidates": [
    { "url": "...", "teams": "A vs B", "time": "...",
      "league": "...", "broadcaster": "..." }
  ],
  "pages_checked": N,
  "notes": "..."
}
```

STOPPING CRITERIA:

- All visible date/section tabs have been inspected, OR
- 8 tool calls have been made without new candidates, OR
- More than 20 candidates already found.

B.4 Hosting-Page Agent Prompt

Listing B.4: Hosting-page agent system prompt (final version).

ROLE:

You are a hosting-page extraction specialist for illegal streaming evidence collection. Your task is to extract stream evidence (m3u8/mpd URLs) from a host page that embeds a video player.

OBJECTIVE:

Produce verified stream evidence from this hosting page. If no stream can be extracted at host level, identify the embedded player URL for fallback analysis.

TOOLS AVAILABLE:

- inspect_hosting: structural analysis including server buttons, player containers, iframe list, and network signals.
- interact: activate server buttons, click play, dismiss popups. Strategies: selector | text_content | xpath | coordinates | js_click.
- harvest: capture m3u8/mpd/mp4 from network events, player APIs, and performance resource entries.
- get_media_state: check if player is actually playing (vs paused or buffering).
- screenshot: confirm each state transition visually.
- navigate: handle redirect chains and server switches.

REACT LOOP PROCEDURE:

OBSERVE: call inspect_hosting to understand the current host page.

THINK:

- Identify server buttons (labeled serv1, serv2, etc.).
- Identify player containers and iframe srcs.
- Check whether network signals already contain m3u8 patterns.

ACT:

- If m3u8 already visible in network: call harvest immediately.
- Otherwise: activate the primary server button via interact.

OBSERVE: call screenshot to confirm the click effect.

- If a popup/new tab appeared: close it and refocus.
- If player changed state: call get_media_state.

ACT: call harvest after confirmed state transition.

REFLECT:

- If harvest returned m3u8 URLs: compile evidence and stop.
- If not: try next server button.
- If all servers exhausted: return embed_url for fallback.

REPEAT until streams found or all servers exhausted.

POPUP HANDLING:

After any interact call, check if a new tab or popup appeared.

If yes: close the new context and retry the original interaction using a different strategy (text_content or js_click).

OUTPUT FORMAT:

```
{
  "streams": ["https://...m3u8", "..."],
  "embed_url": "https://...",
  "server_tried": ["serv1", "serv2"],
  "evidence_screenshots": ["url1", "url2"],
  "failure_reason": null | "popup_blocked" | "no_stream_found" | ...
}
```

STOPPING CRITERIA:

- Confirmed m3u8/mpd URLs found, OR
- All visible server options exhausted, OR
- 12 total tool calls reached.

B.5 Embedded-Page Agent Prompt

Listing B.5: Embedded-page agent system prompt (final version).

ROLE:

You are an embedded-player extraction specialist. Your task is to extract streaming evidence from a minimal iframe-based player page.

OBJECTIVE:

Activate the player (if not auto-playing) and capture m3u8/mpd URLs from the page network traffic and player internals.

TOOLS AVAILABLE:

- inspect_embedded: analyze frame structure, player type (albaplayer, clappr, hls.js, videojs), play button presence, auto-play status.
- interact: click the play button or dismiss any overlay.
Strategies: selector | text_content | coordinates | js_click.
- harvest: capture stream URLs from network events, performance resource entries, and player source attributes.
- navigate: follow embedded redirects if iframe resolves to a different origin.
- screenshot: visual confirmation of player state.

REACT LOOP PROCEDURE:

OBSERVE: call inspect_embedded to assess player readiness.

THINK:

- Is the player auto-playing? If yes: call harvest directly.
- Is a play button visible? If yes: click it first.
- Are there ad overlays blocking the play button?

ACT:

- Dismiss any visible ad overlay using interact.
- Click the play button.
- Wait 2 seconds for network requests to generate.

OBSERVE: call screenshot to confirm player state.

ACT: call harvest to capture all stream-related requests.

REFLECT:


```
- If m3u8 found: return evidence and stop.
- If not: inspect frame tree for deeper nested context.
- Try navigating directly to player src if frame access is blocked.
REPEAT up to 3 interaction attempts.
```

PLAYER ACTIVATION PATTERN (detailed):

1. inspect_embedded -> identify player container CSS selector or data-attribute.
2. interact with strategy="selector", target=".play-btn" (or coordinates if selector fails).
3. harvest after 2-3s wait.
4. If no stream: try js_click strategy on the player container itself.
5. harvest again.

OUTPUT FORMAT:

```
{
  "streams": ["https://...m3u8"],
  "player_type": "albaplayer | clappr | hls.js | unknown",
  "play_required": true | false,
  "evidence_screenshots": ["url"],
  "failure_reason": null | "stream_masked" | "no_play_button" | ...
}
```

B.6 Provider and Email Prompt Contract

Listing B.6: Provider-enrichment and email prompt contract (final version).

ROLE:

You prepare provider context and a reviewable takedown-email draft from already collected stream evidence. You are not a browser agent.

OBJECTIVE:

Convert extracted streams, screenshots, page URLs, and provider lookup results into a structured review package that a human can inspect.

INPUTS:

- pipeline_result,
- stream URLs and source pages,
- screenshots,
- provider lookup rows,
- rights-owner context,
- run timestamp and target URL.

PROCEDURE:

1. Group streams by hostname/provider when possible.
2. Preserve the original stream URL exactly as observed.
3. Attach screenshot references and run identifiers.
4. Include provider, IP, country, and abuse-contact fields when found.
5. If abuse contact is missing, keep provider context and mark contact

status explicitly as missing.

6. Draft a concise message for human review. Do not send it.

OUTPUT FORMAT:

```
{
  "provider_groups": [
    {
      "provider": "...",
      "hosts": [...],
      "streams": [...],
      "country": "...",
      "abuse_contact": null | "...",
      "screenshots": [...]"
    }
  ],
  "draft_email": {
    "subject": "...",
    "body": "...",
    "requires_human_approval": true
  }
}
```

GUARDRAILS:

- Do not invent provider names or contacts.
- Do not claim legal certainty from technical evidence alone.
- Do not run when no stream or embedded evidence exists.
- Keep missing data visible instead of filling it with assumptions.

Detailed Agent Responsibility Views

The main architecture chapter keeps the agent design compact. This appendix keeps the additional responsibility views that are useful for technical review but would make the main chapter too heavy. Each view follows the same pattern: input context, agent responsibility, output contract, and guardrails.

C.1 Orchestrator

The orchestrator owns the route graph. It does not inspect websites directly. Its value is control: it builds handoffs, keeps state, aggregates evidence, and decides whether downstream provider and email steps should run.

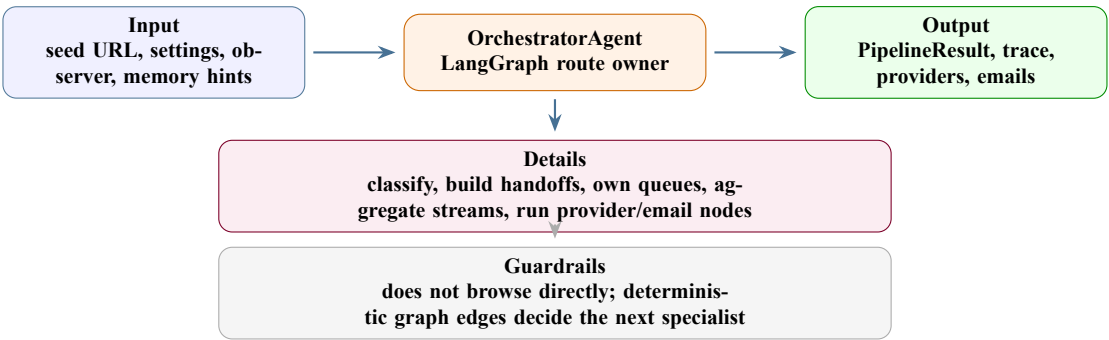


Figure C.1: Orchestrator-agent design card.

Responsibility	Detail
Input control	Receives the seed URL, settings, memory hints, browser profile, and observer handle.
Route ownership	Calls classification first, then selects landing, hosting, or embedded specialists.
State aggregation	Combines specialist outputs into one pipeline result.
Guardrail	Does not browse directly and does not let specialists call each other outside the route graph.

Table C.1: Orchestrator responsibility checklist.

C.2 Classification Agent

The classification agent has the smallest browser budget because its task is narrow. It must identify the page role, not solve the whole investigation. This keeps the first decision fast and prevents the system from wasting browser actions on the wrong page type.

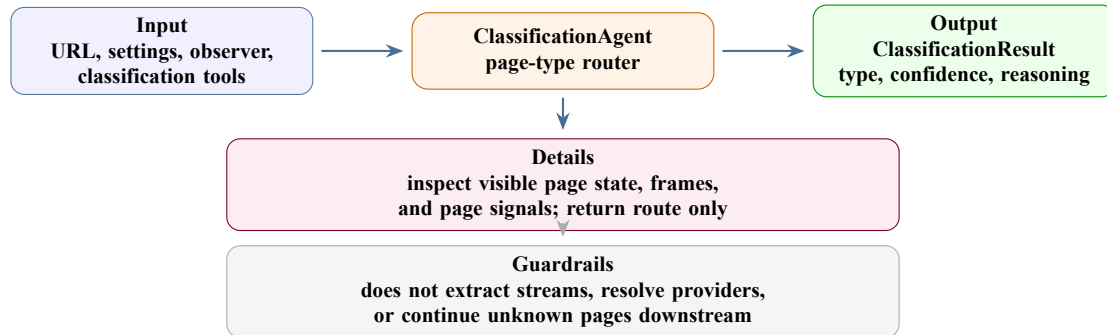


Figure C.2: Classification-agent design card.

Responsibility	Detail
Signals used	Title, visible text, links, iframes, player hints, screenshots, and page structure.
Output	ClassificationResult with page type, confidence, and route reason.
Failure mode	Mixed-role pages can contain both listing and player signals.
Guardrail	The classifier does not click, navigate, or harvest streams.

Table C.2: Classification-agent responsibility checklist.

C.3 Landing-Page Agent

The landing-page agent is responsible for discovery. It should not try to activate players unless the page clearly becomes a hosting page. Its job is to inspect event cards, tabs, schedules, and links, then pass good candidates to the next stage.

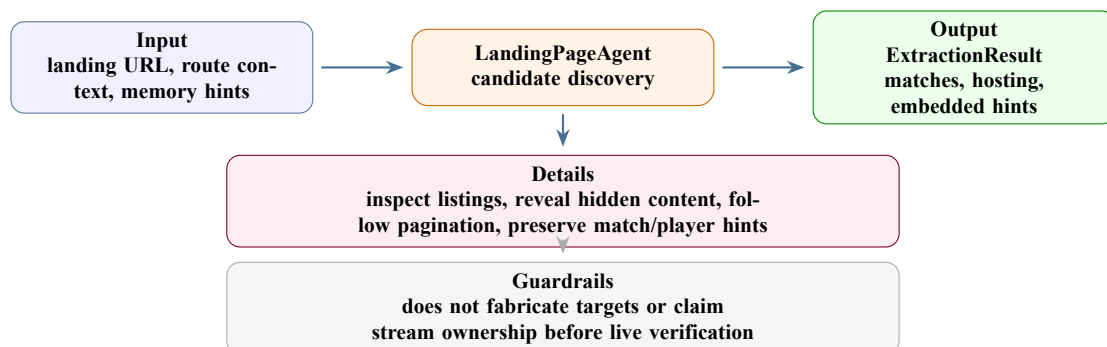


Figure C.3: Landing-page-agent design card.

Responsibility	Detail
Evidence target	Match cards, event links, channel pages, watch buttons, host candidates, and visible broadcaster hints.
Interaction policy	Scroll, open date tabs, reveal hidden sections, and avoid unrelated navigation.
Output	Candidate URLs ranked with match or page context.
Guardrail	Stop after visible discovery is covered; do not crawl the entire domain.

Table C.3: Landing-agent responsibility checklist.

C.4 Hosting-Page Agent

The hosting-page agent performs the most fragile browser work. It must activate server buttons and player controls while handling popups and redirects. It also decides when a page should be handed to the embedded agent.

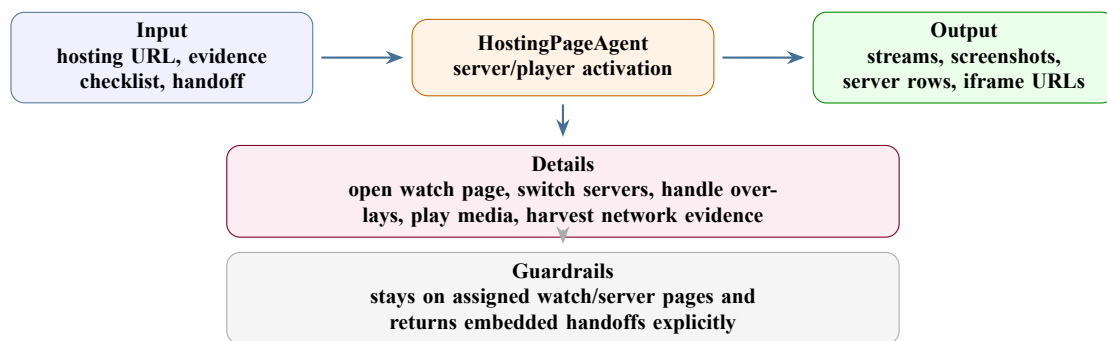


Figure C.4: Hosting-page-agent design card.

Responsibility	Detail
Evidence target	Player activation, stream candidates, iframe URLs, server attempts, and screenshots.
Interaction policy	Try server buttons deliberately, close popups, restore focus, and verify player state before harvesting.
Output	Streams when available, otherwise embedded-player handoff or clear failure reason.
Guardrail	Do not mark success from a screenshot alone; require stream, embedded, or blocker evidence.

Table C.4: Hosting-agent responsibility checklist.

C.5 Embedded-Page Agent

The embedded-page agent works on direct player or iframe URLs. It is often the fallback when a hosting page exposes an iframe but does not return media evidence at the host level.

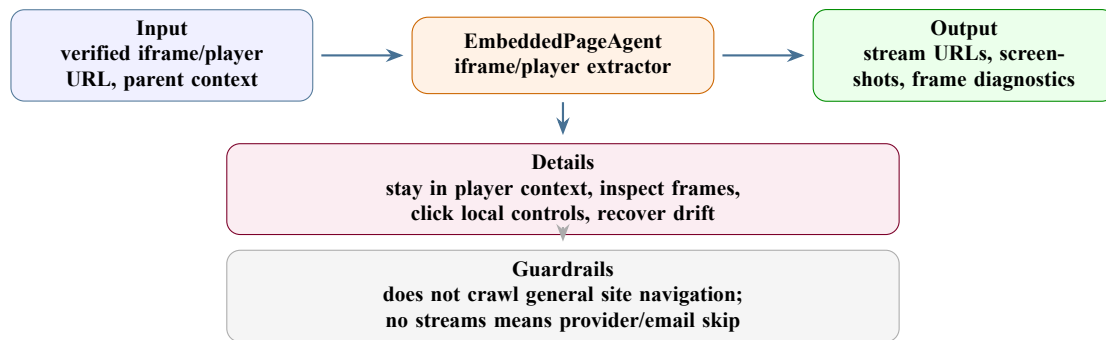


Figure C.5: Embedded-page-agent design card.

Responsibility	Detail
Evidence target	Player runtime, nested frame URLs, manifests, media requests, and visual playback state.
Interaction policy	Inspect first, click play only when needed, wait for network activity, then harvest.
Output	Stream evidence, player type, screenshots, or reason why playback was blocked.
Guardrail	Do not crawl away from the assigned embedded/player context.

Table C.5: Embedded-agent responsibility checklist.

C.6 Provider and Email Stage

Provider analysis and email drafting run only after stream or embedded evidence exists. They are downstream packaging stages, not browser investigators. This separation avoids mixing technical extraction with review communication.

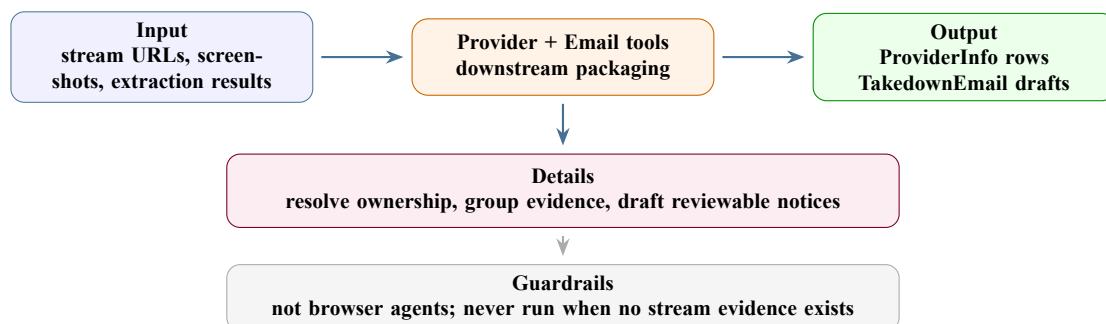


Figure C.6: Provider-analysis and email-generation design card.

C.7 Responsibility Matrix

Responsibility	Detail
Input	Stream URLs, source pages, screenshots, provider lookups, and rights context.
Output	Provider rows and email drafts that remain subject to human approval.
Failure mode	Provider can be known while abuse contact is missing.
Guardrail	Do not run when no stream or embedded evidence exists, and do not invent missing provider details.

Table C.6: Provider/email responsibility checklist.

Table C.7: Detailed agent responsibility matrix.

Agent or stage	Owns	Does not own	Main output
Orchestrator	Route graph, handoffs, state, final aggregation.	Direct browser clicks, provider assumptions, manual decisions.	Pipeline result.
Classification	Page type and route reason.	Navigation, extraction, provider lookup.	Classification result.
Landing	Candidate discovery from schedules and event lists.	Player activation and provider analysis.	Candidate list.
Hosting	Server/player interaction and host-level stream extraction.	Whole site crawling and email generation.	Streams or embedded handoff.
Embedded	Player-only and iframe-level extraction.	Landing-page discovery and unrelated redirects.	Streams or player failure.
Provider/email	Infrastructure enrichment and reviewable draft.	Browser investigation and final human approval.	Provider rows and draft email.

C.8 Why These Boundaries Matter

The boundaries were introduced because early experiments mixed too many responsibilities. A single broad agent could classify a page, click a link, open a popup, harvest too early, and still produce a confident final message. That made failures difficult to understand. Splitting the runtime into bounded agents made each failure more concrete:

- wrong page type means classification or handoff issue;
- no candidate links means landing extraction issue;
- no player activation means hosting interaction issue;

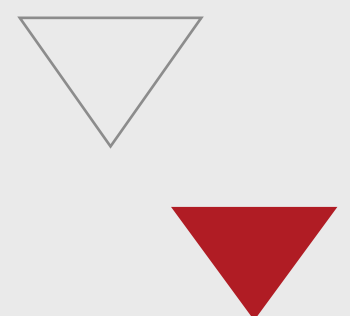
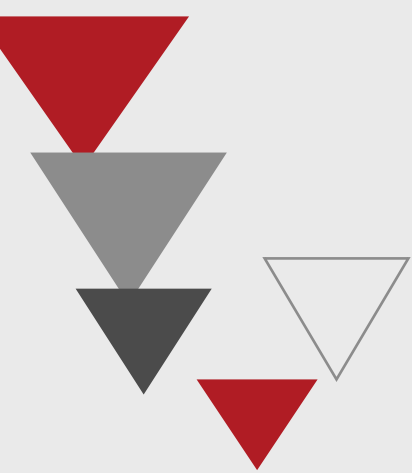
- no manifest after iframe handoff means embedded extraction issue;
- provider found but no abuse contact means enrichment coverage issue;
- high cost with no new evidence means tool-budget or prompt issue.

This responsibility model also supports future work. New specialists can be added without rewriting the whole pipeline, and regression tests can target one stage at a time.

Bibliography

- [1] OpenRouter, *Models API*, technical reference, 2026. Available: <https://openrouter.ai/api/v1/models>
- [2] Google, *Gemini API Rate Limits*, Google AI for Developers, 2026. Available: Google AI for Developers.
- [3] Google, *Gemini Developer API Pricing*, Google AI for Developers, 2026. Available: Google AI for Developers.
- [4] Google, *Context Caching in the Gemini API*, Google AI for Developers, 2026. Available: Google AI for Developers.
- [5] Google Chrome Developers, *Puppeteer API Reference*, API reference, 2026. Available: <https://pptr.dev>
- [6] Microsoft, *Playwright Browser Automation Guide*, developer reference, 2026. Available: Playwright.
- [7] Chrome DevTools Protocol, *Chrome DevTools Protocol*, protocol reference, 2026. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [8] WHATWG, *HTML Living Standard*, Web Hypertext Application Technology Working Group, 2026. Available: <https://html.spec.whatwg.org/>
- [9] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: <https://fastapi.tiangolo.com>
- [10] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: Next.js.
- [11] Model Context Protocol, *Model Context Protocol Specification*, protocol specification, 2024. Available: <https://modelcontextprotocol.io/specification/2024-11-05/basic/index>
- [12] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: n8n.
- [13] Microsoft, “Azure Service Bus Messaging Service,” *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>
- [14] IPinfo, *IPinfo API*, developer reference, 2026. Available: <https://ipinfo.io/developers/ipinfo-api>
- [15] ICANN, *Registration Data Access Protocol (RDAP)*, technical reference, 2026. Available: <https://www.icann.org/rdap/>
- [16] Cloudflare, *How Cloudflare Challenges Work*, technical reference, 2026. Available: <https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/>

- [17] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: <https://www.rfc-editor.org/rfc/rfc8216>
- [18] ISO/IEC, *Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*, ISO/IEC 23009-1, 2022.
- [19] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: <https://github.com/gorhill/uBlock>



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn – E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax : +216 70 685 454